

Deep Learning: Sequence Models

Part – 1

Dr. Oybek Eraliev,

Department of Computer Engineering

Inha University In Tashkent.

Email: oybekeraliev7@gmail.com



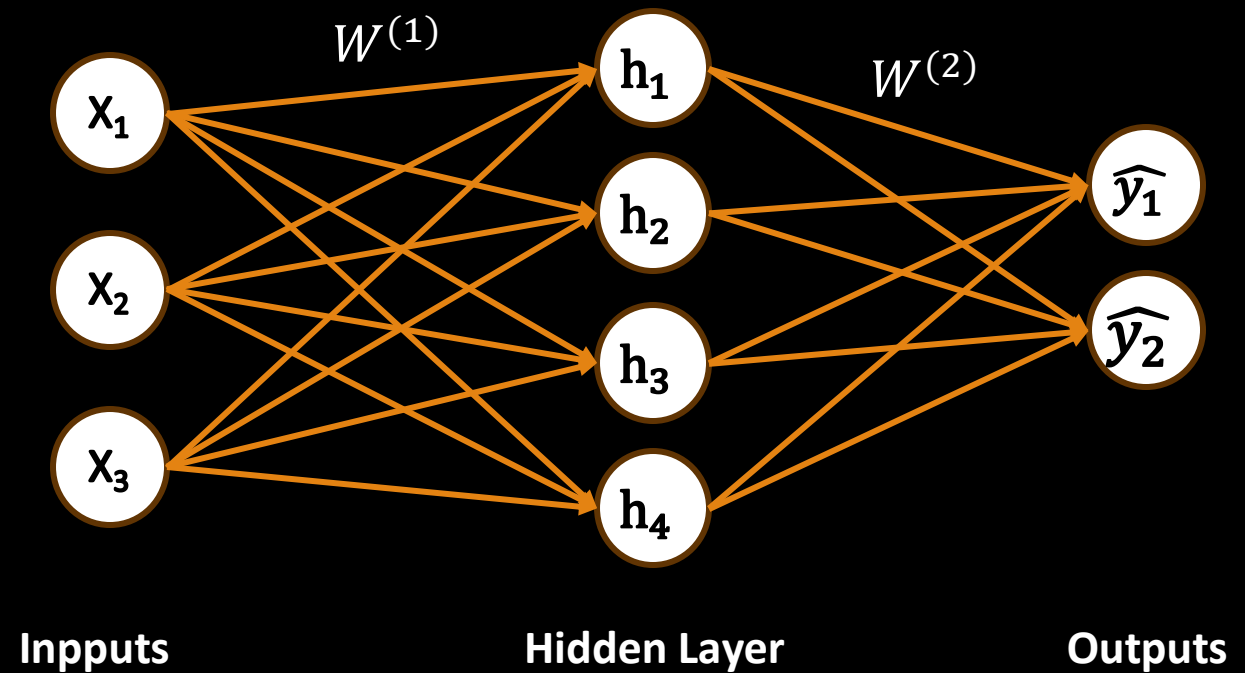
Content

- **Recurrent Neural Network (RNN)**
- **Long Short-Term Memory (LSTM)**
- **Gated Recurrent Unit Network (GRUs)**
- **Implementation of RNN, GRU and LSTM**

Recurrent Neural Network

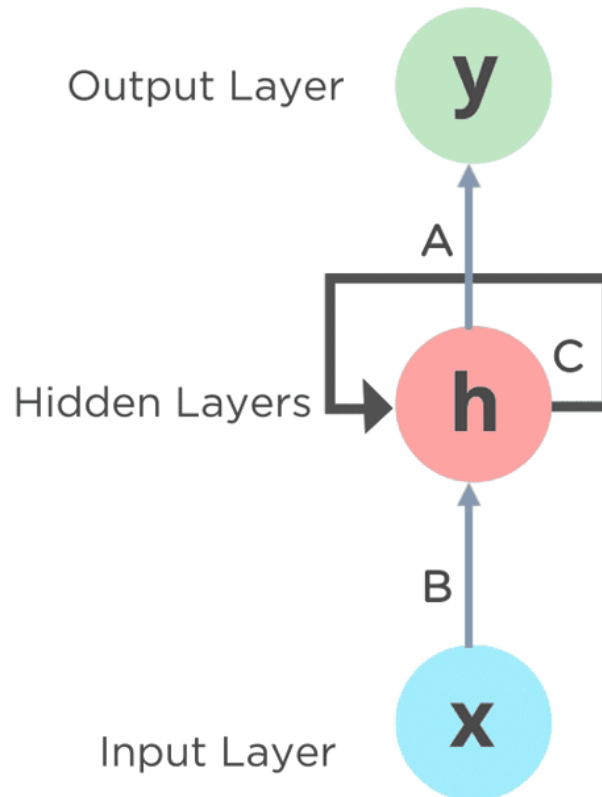
“Vanilla” neural network

- A vanilla neural network (NN) is a type of artificial intelligence (AI) model that is made up of an input layer, an output layer, and multiple hidden layers. The hidden layers are fully connected, meaning that each neuron in one layer is connected to every neuron in the next layer. The output of each layer is then fed through a nonlinear activation function.



Recurrent Neural Network

“Vanilla” neural network

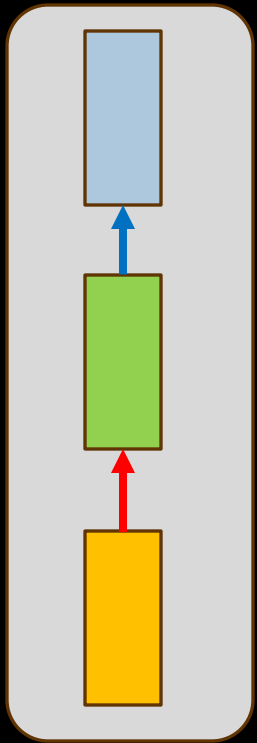


A, B and C are the parameters

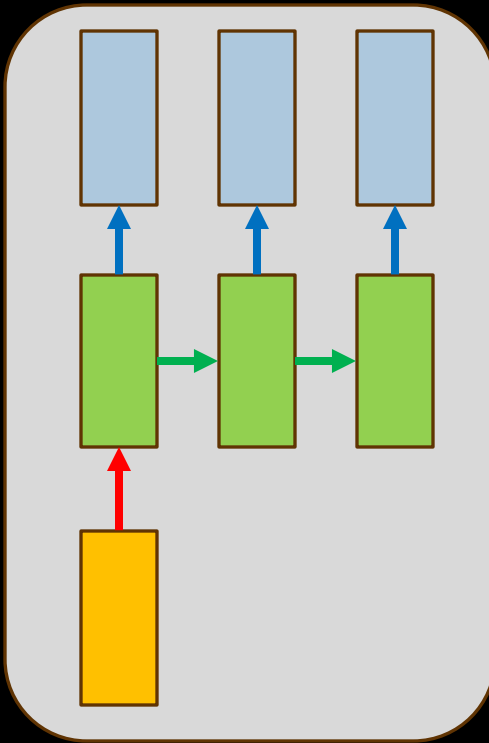
Recurrent Neural Network

Recurrent Neural Network: Process Sequence

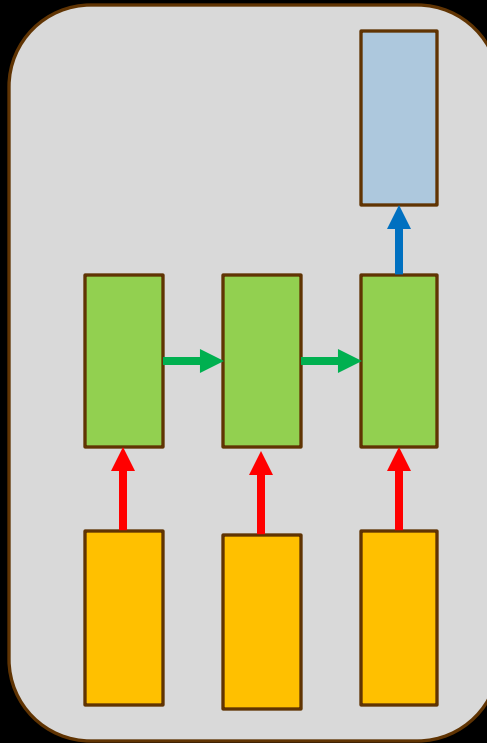
one to one



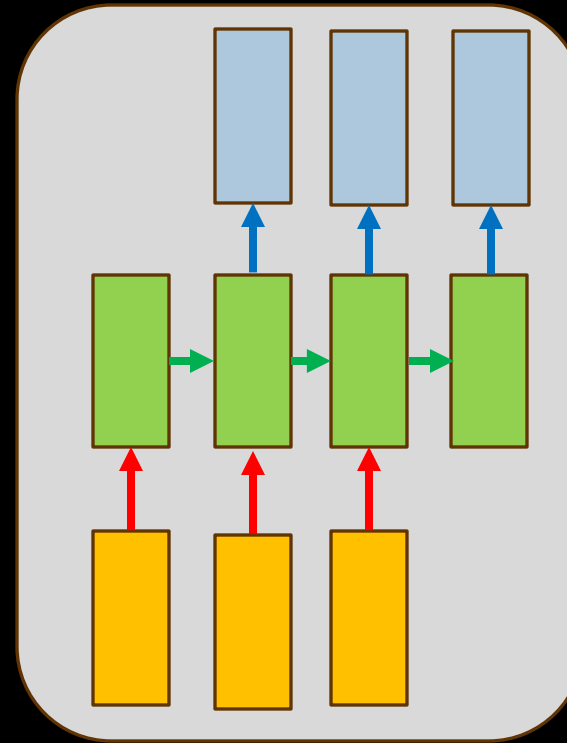
one to many



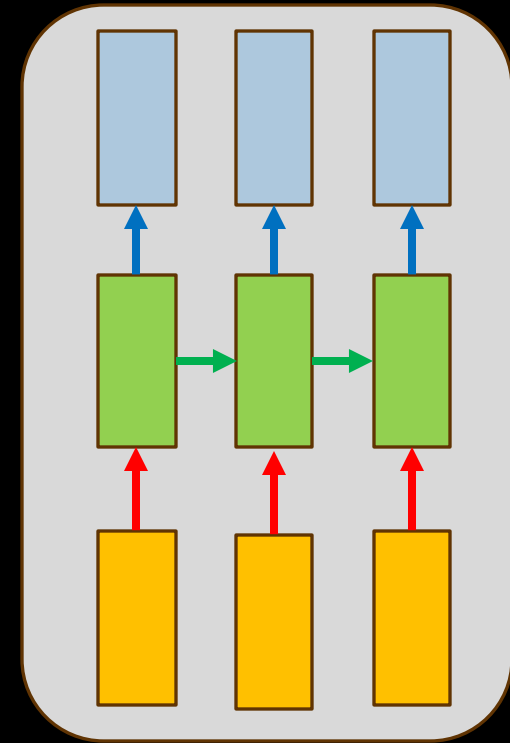
many to one



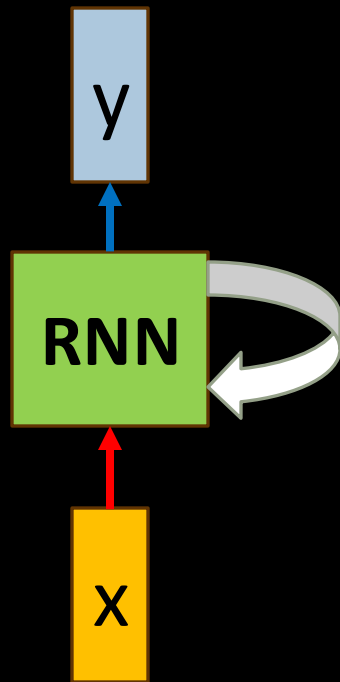
many to many



many to many

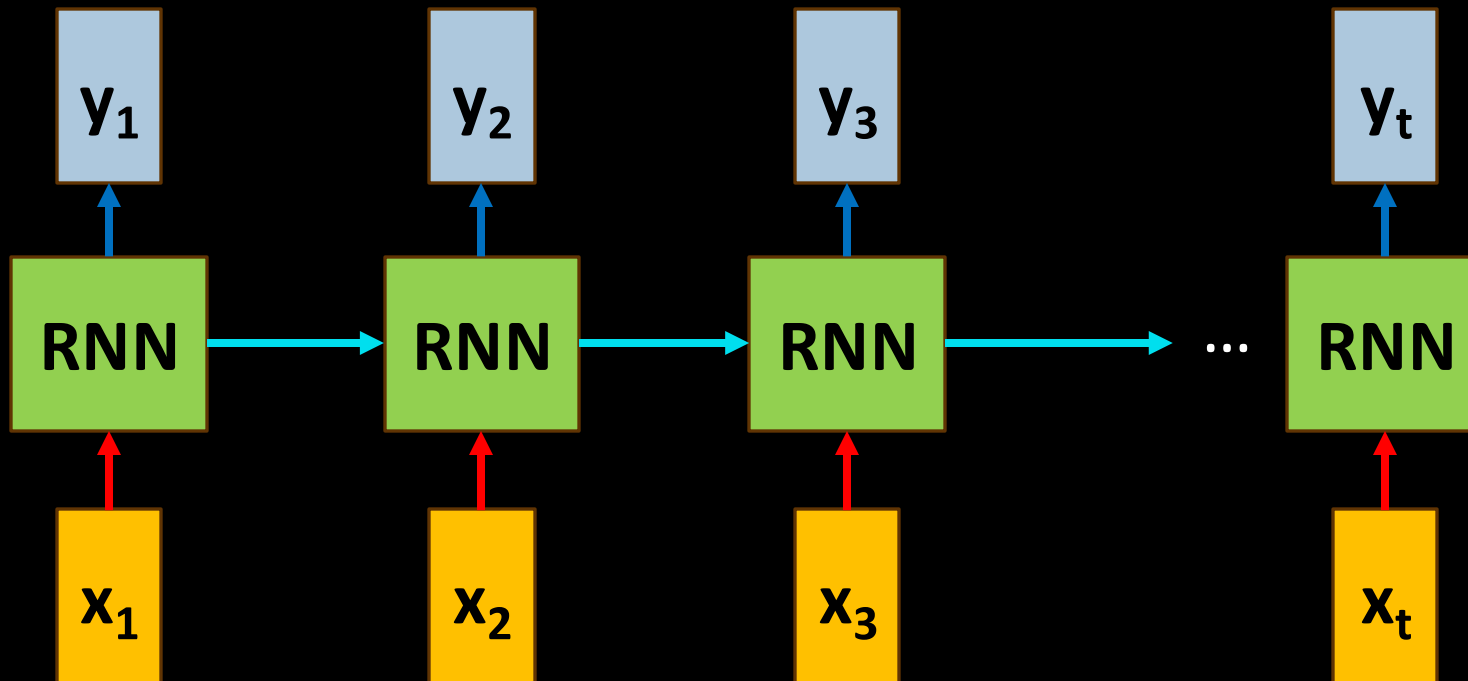


Recurrent Neural Network



Key idea: RNNs have an “**internal state**” that is updated as a sequence is processed

Recurrent Neural Network



Recurrent Neural Network

RNN hidden state update

We can process a sequence of vectors x by applying a recurrence formula at every time step:

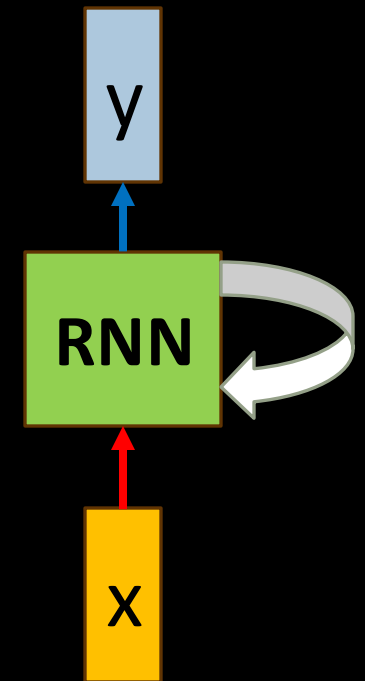
$$h_t = f_w(h_{t-1}, x_t)$$

New state

some function with parameters W

Old state

Input vector at some time step



Recurrent Neural Network

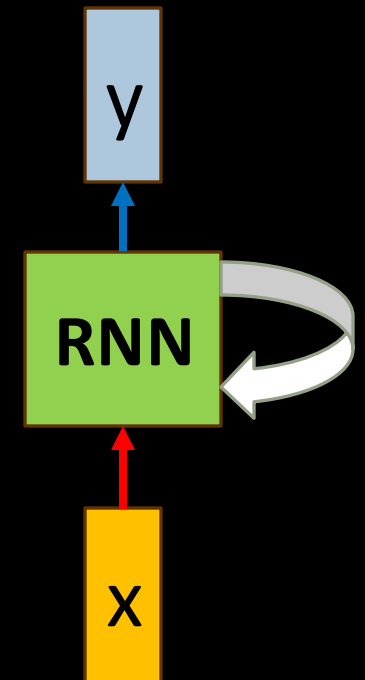
RNN output generation

We can process a sequence of vectors x by applying a recurrence formula at every time step:

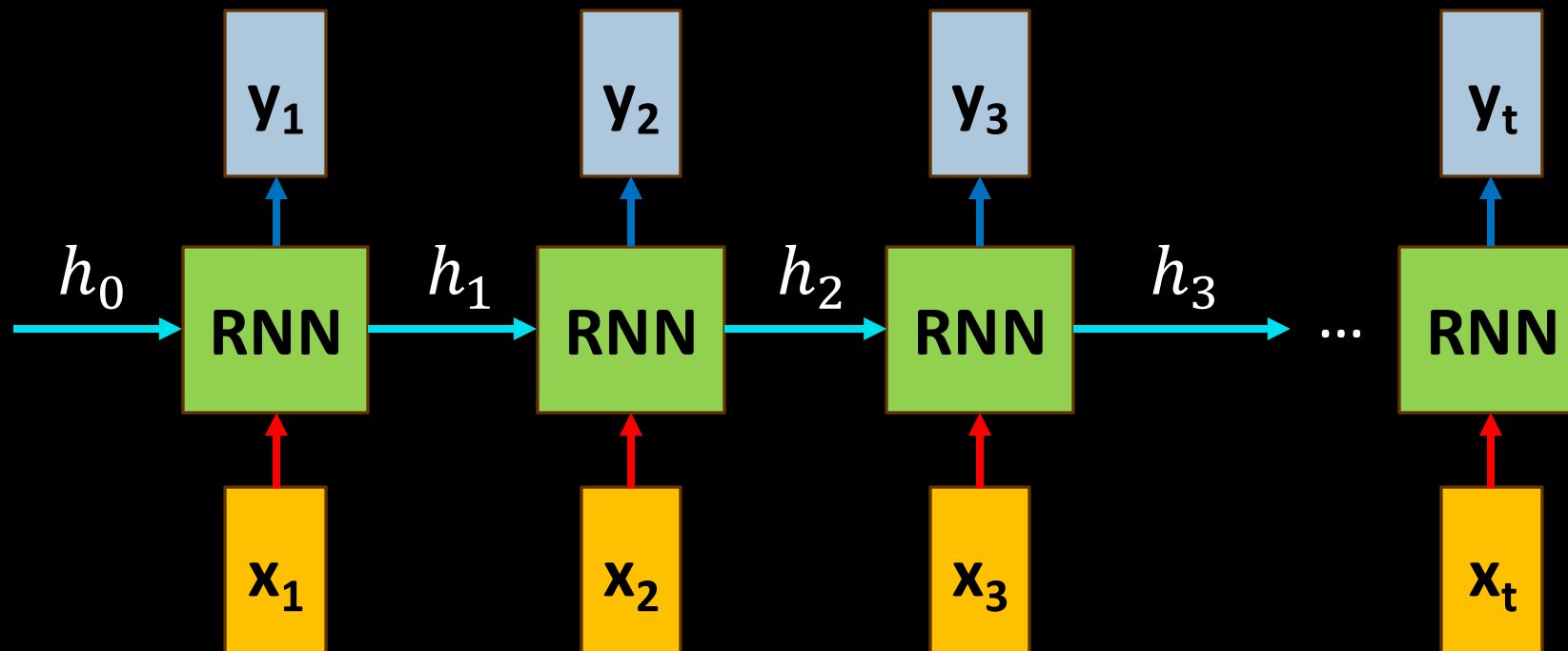
$$y_t = f_{W_y}(h_t)$$

Output New state

another function with
parameters W_y



Recurrent Neural Network

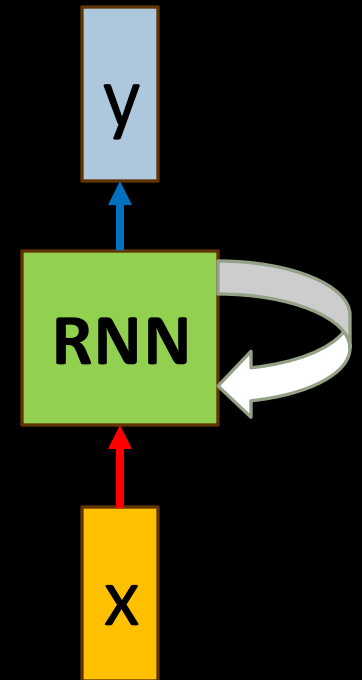


Recurrent Neural Network

We can process a sequence of vectors x by applying a recurrence formula at every time step:

$$h_t = f_w(h_{t-1}, x_t)$$

Notice: the same function and the same set of parameters are used at every time step.



Recurrent Neural Network

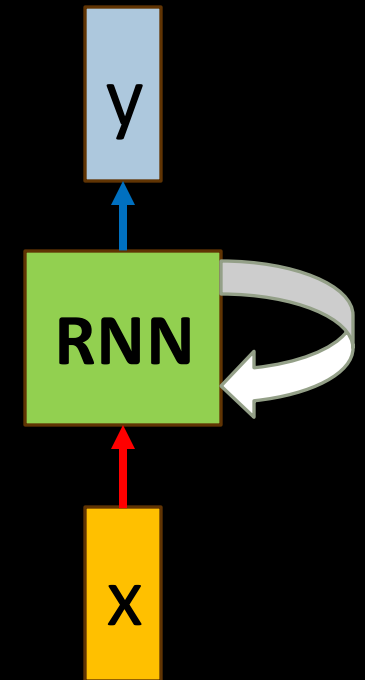
Recurrent Neural Network (Vanilla)

The state consists of a single “hidden” vector h :

$$h_t = f_w(h_{t-1}, x_t)$$

$$h_t = \tanh(W_h h_{t-1}, W_x x_t)$$

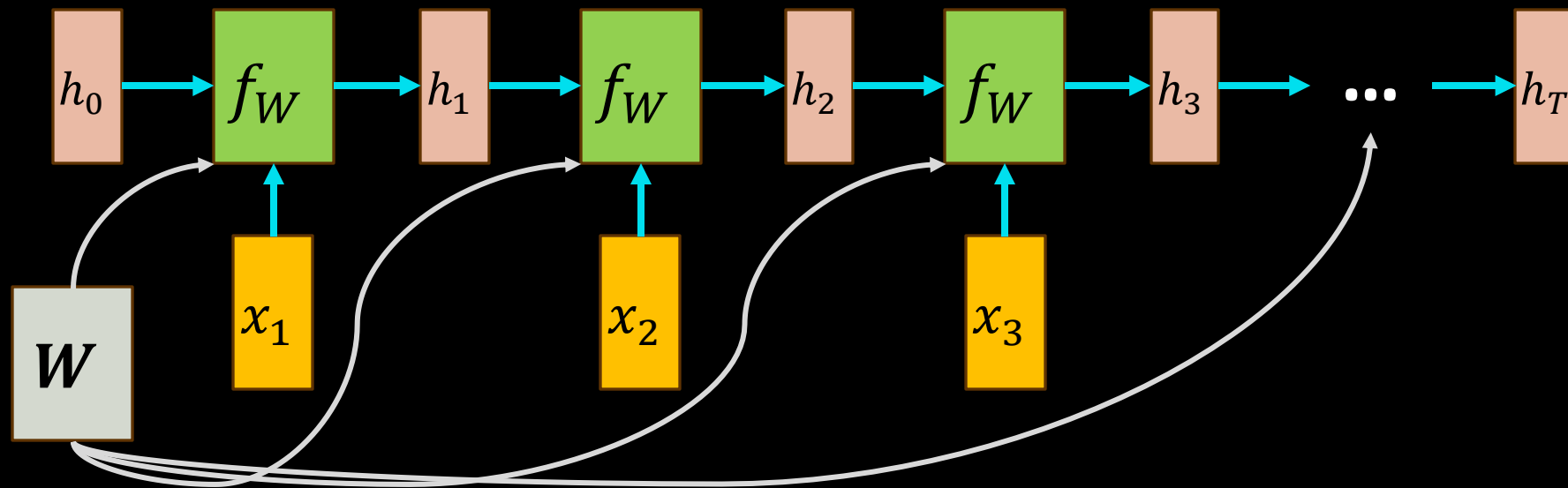
$$y_t = f(W_y, h_t)$$



Recurrent Neural Network

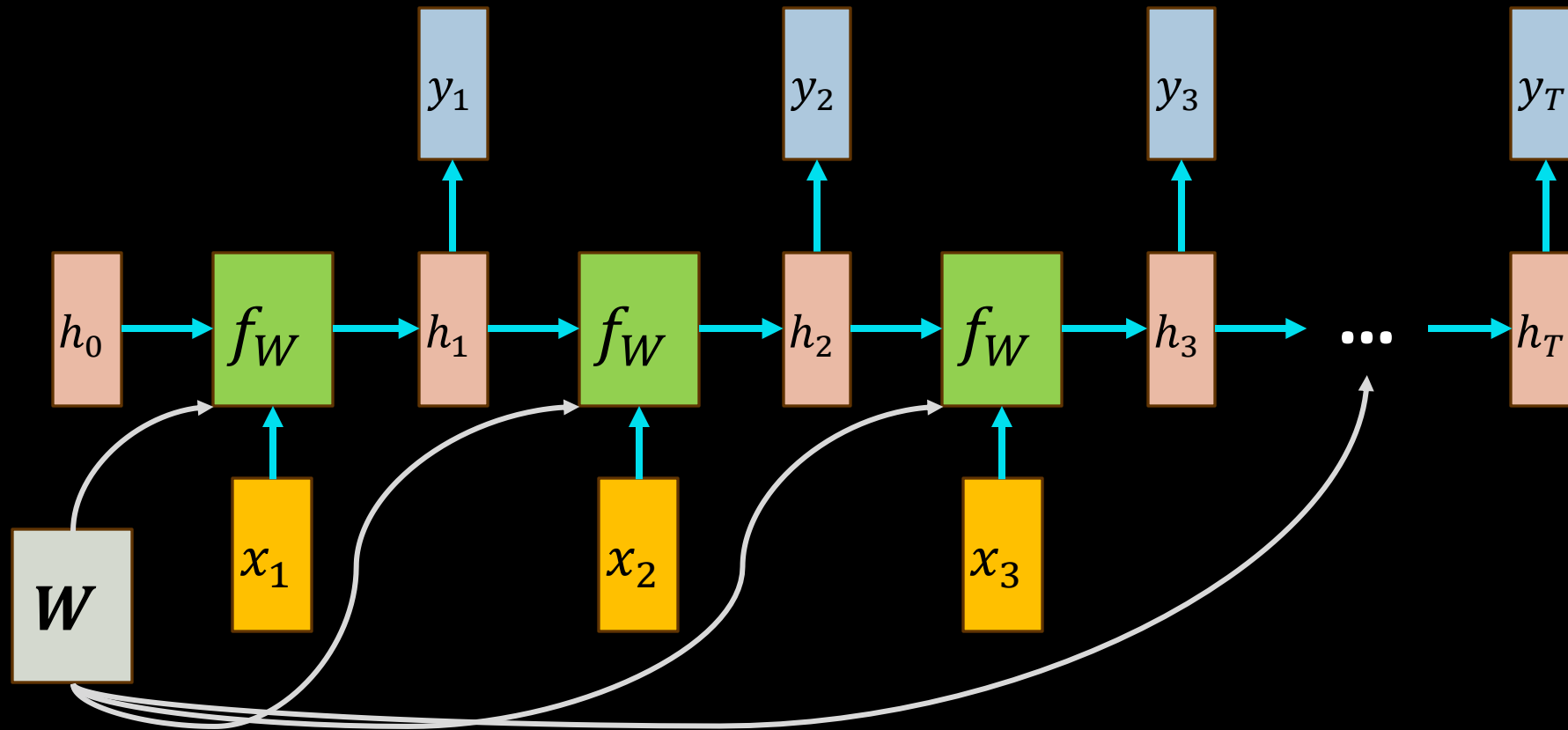
RNN: Computational Graph

Re-use the same weight matrix at every time-step



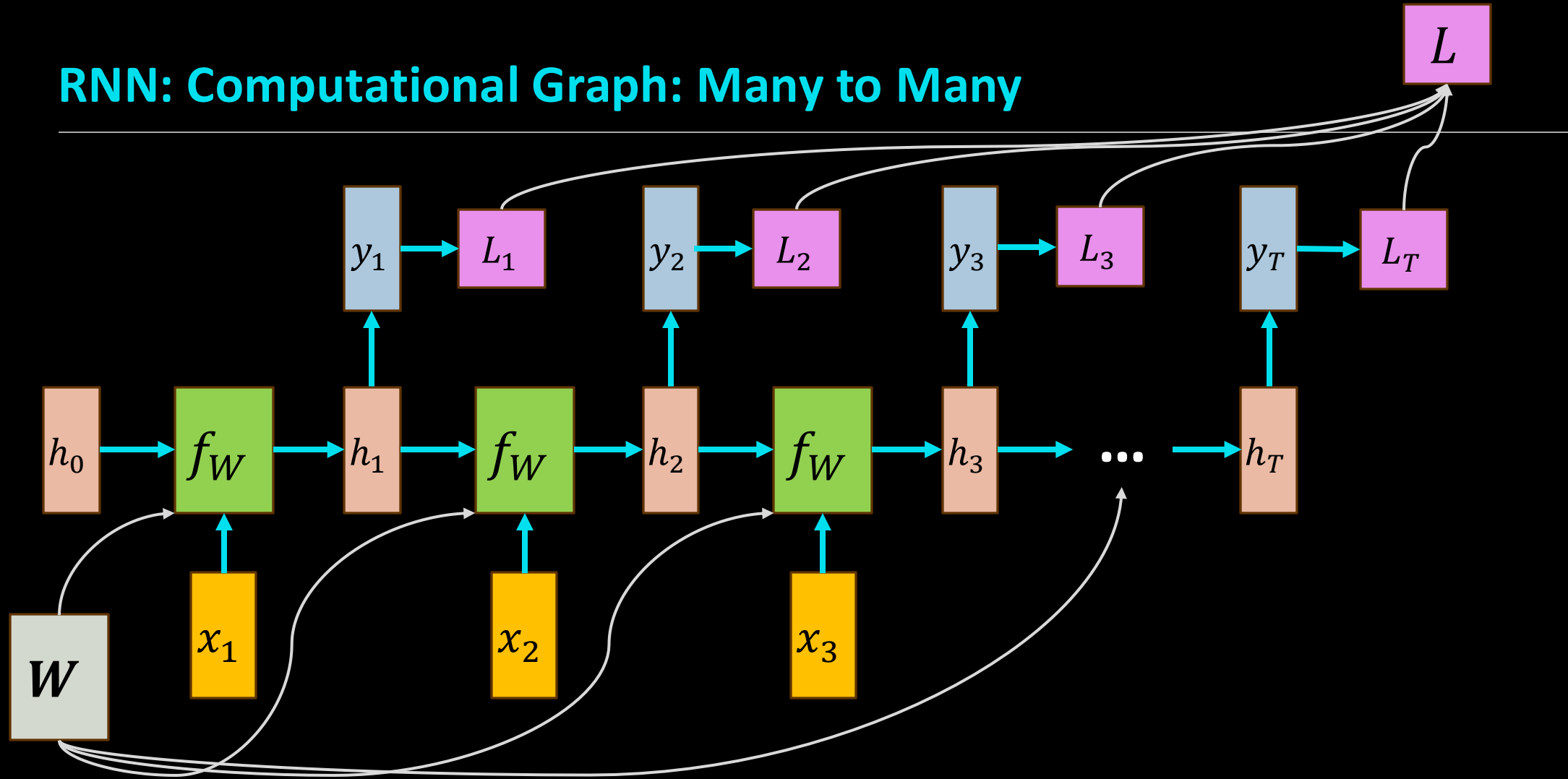
Recurrent Neural Network

RNN: Computational Graph: Many to Many



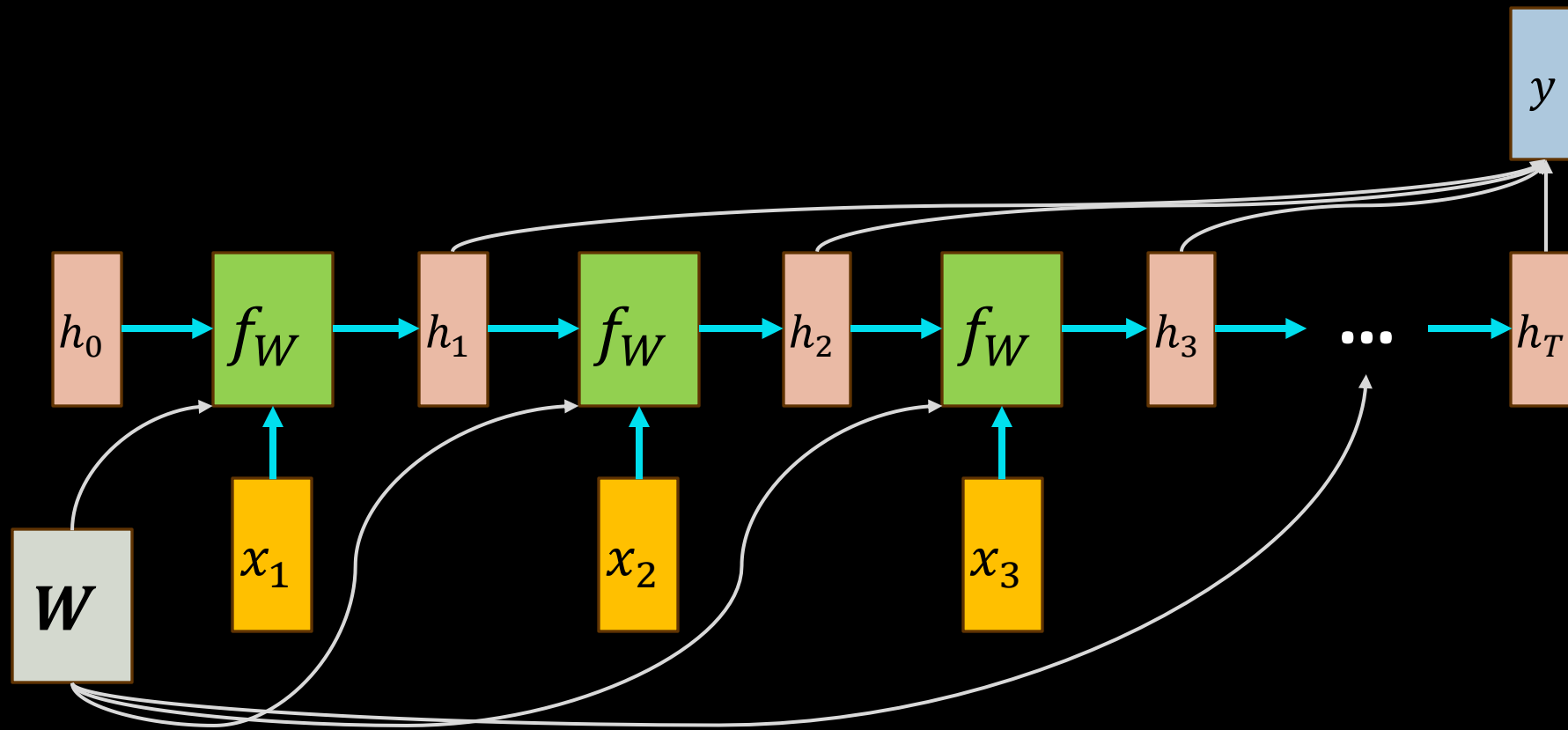
Recurrent Neural Network

RNN: Computational Graph: Many to Many



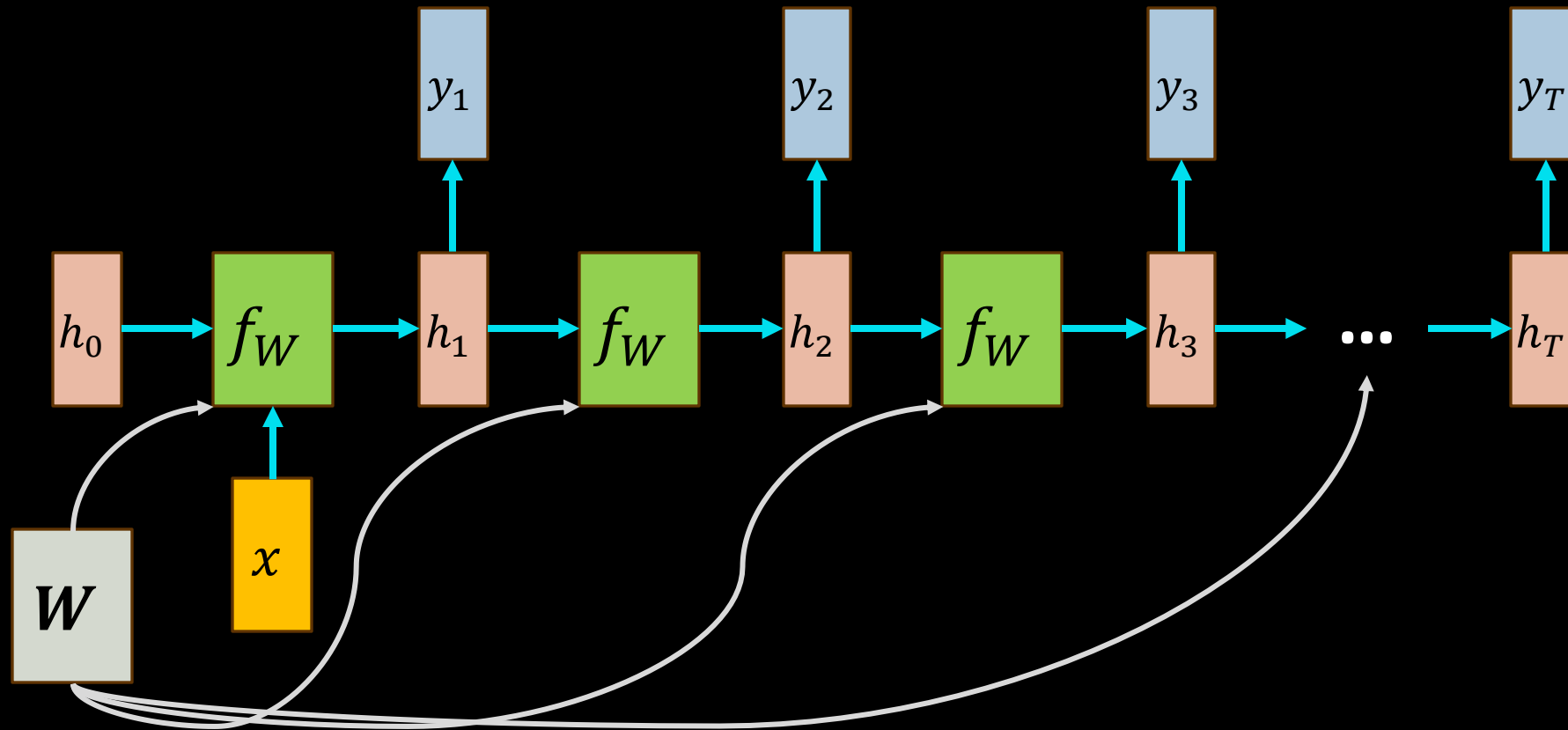
Recurrent Neural Network

RNN: Computational Graph: Many to One



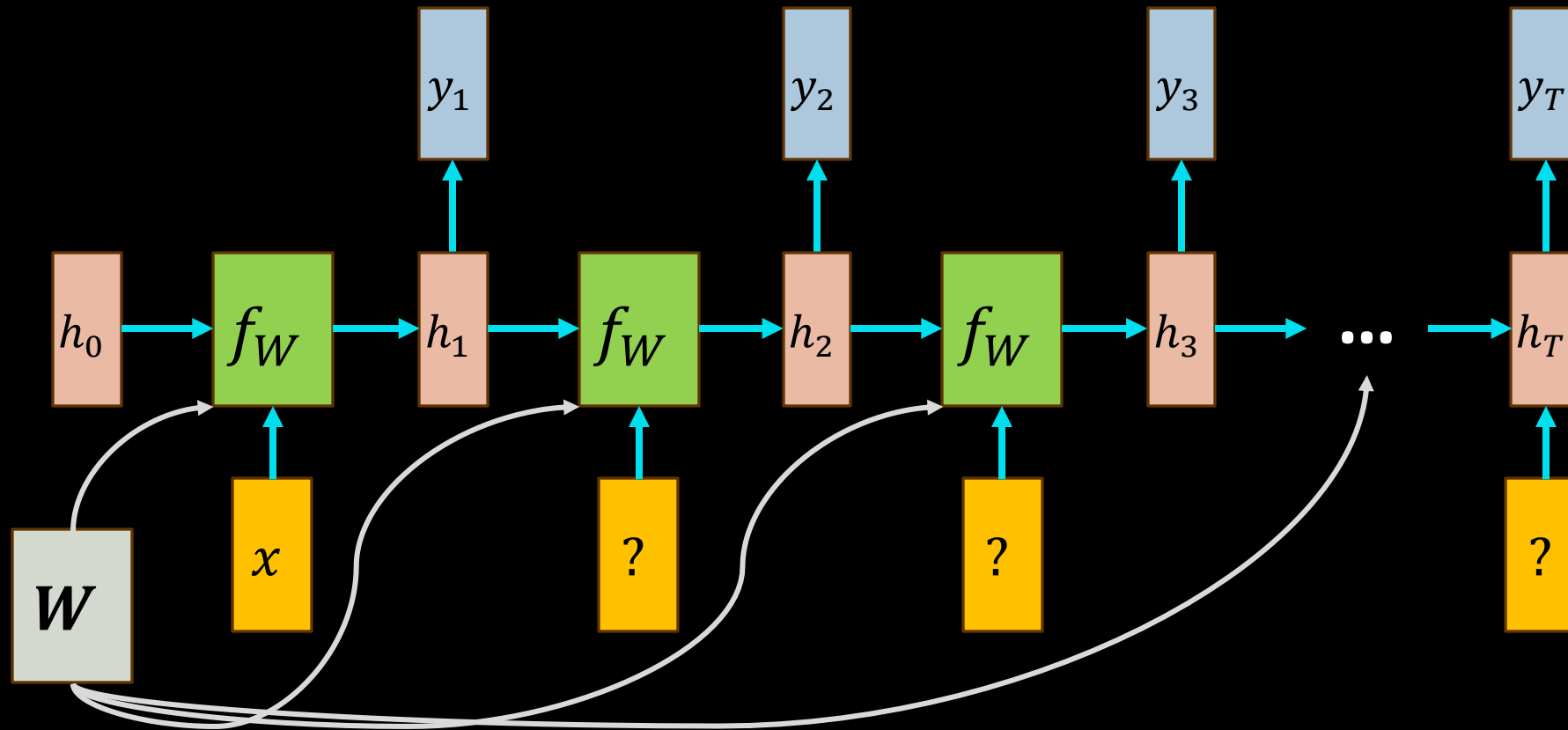
Recurrent Neural Network

RNN: Computational Graph: One to Many



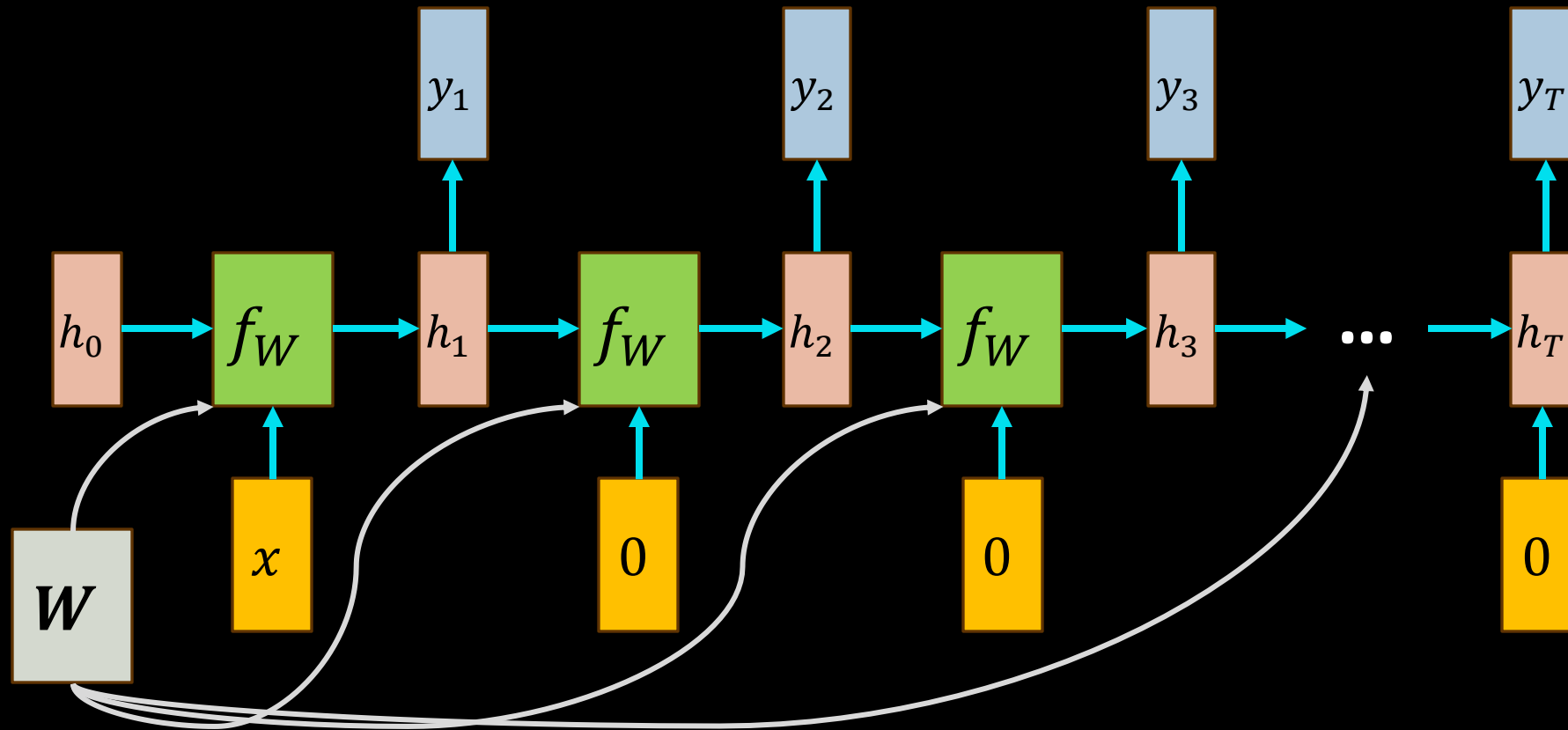
Recurrent Neural Network

RNN: Computational Graph: One to Many



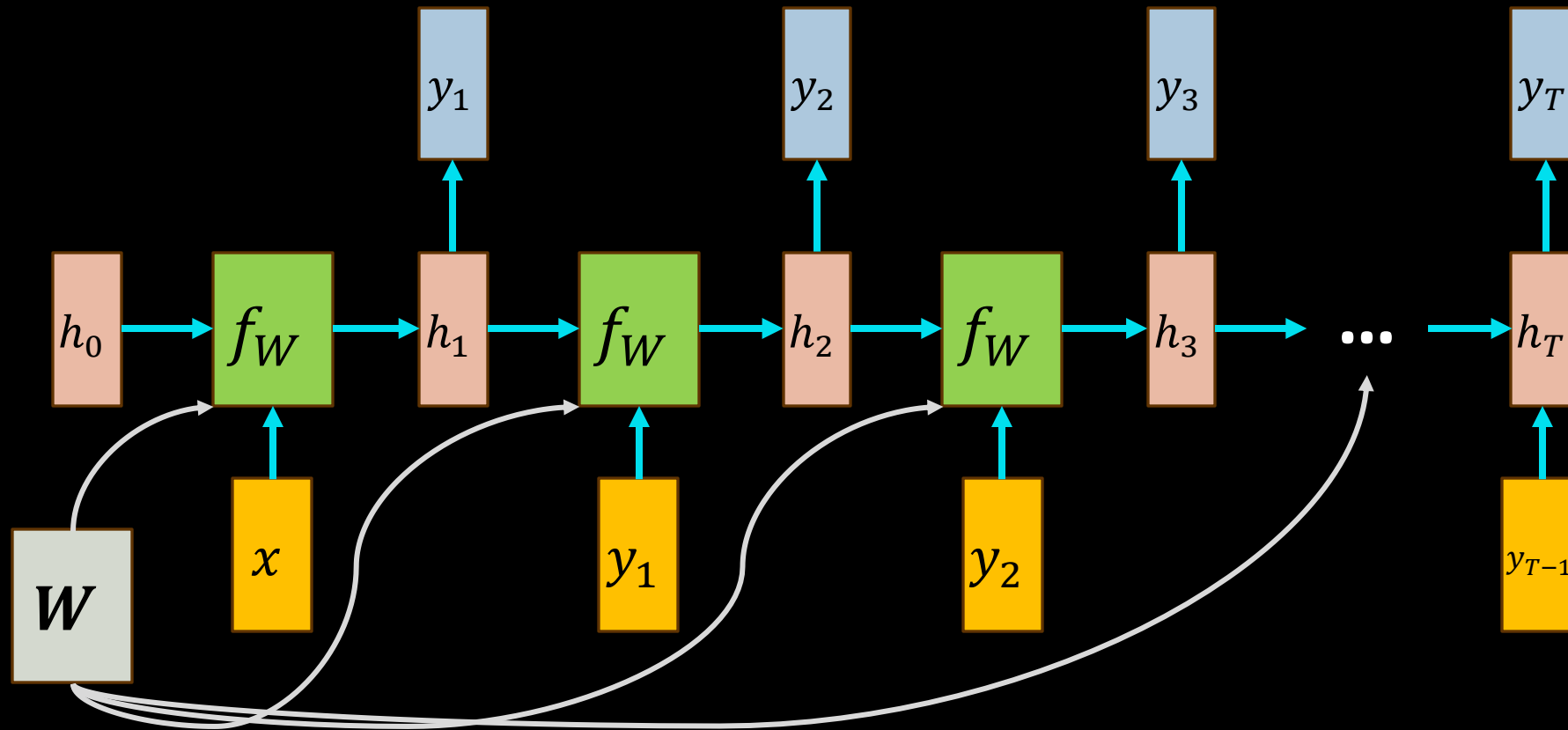
Recurrent Neural Network

RNN: Computational Graph: One to Many



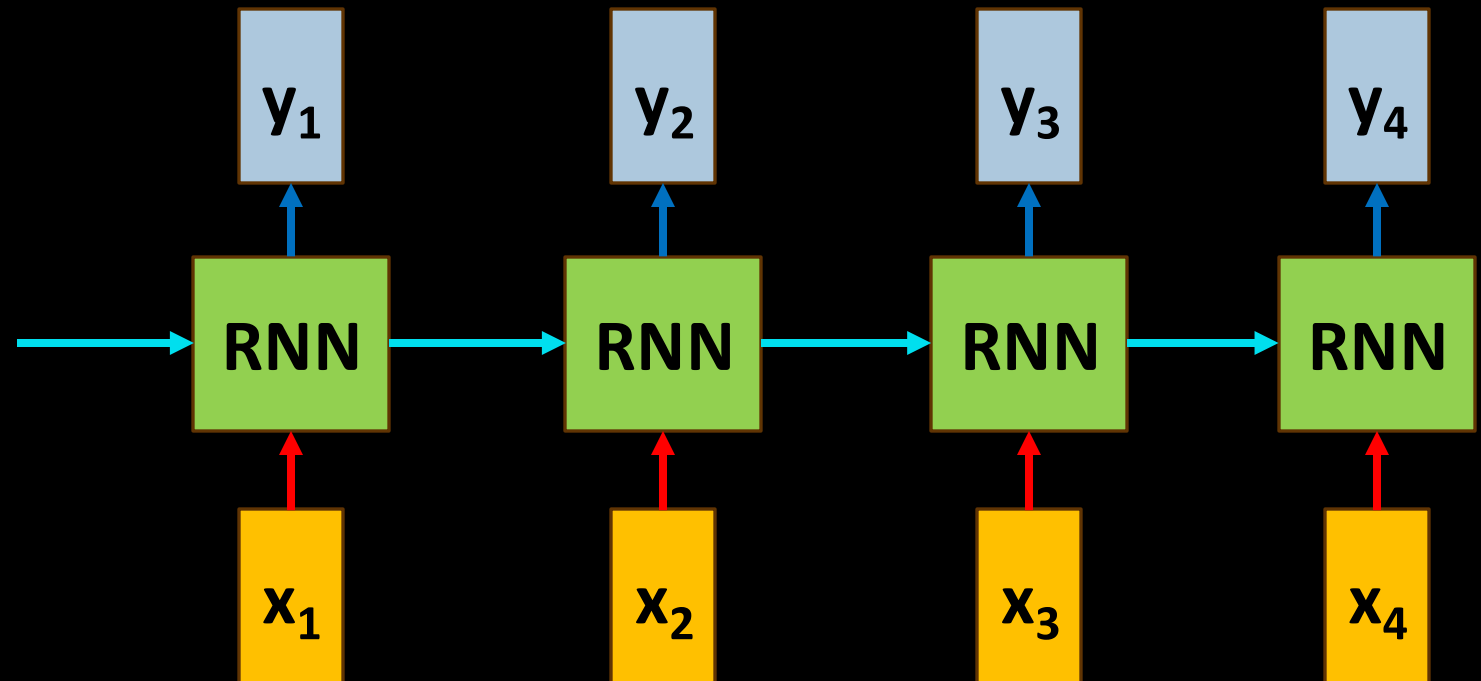
Recurrent Neural Network

RNN: Computational Graph: One to Many



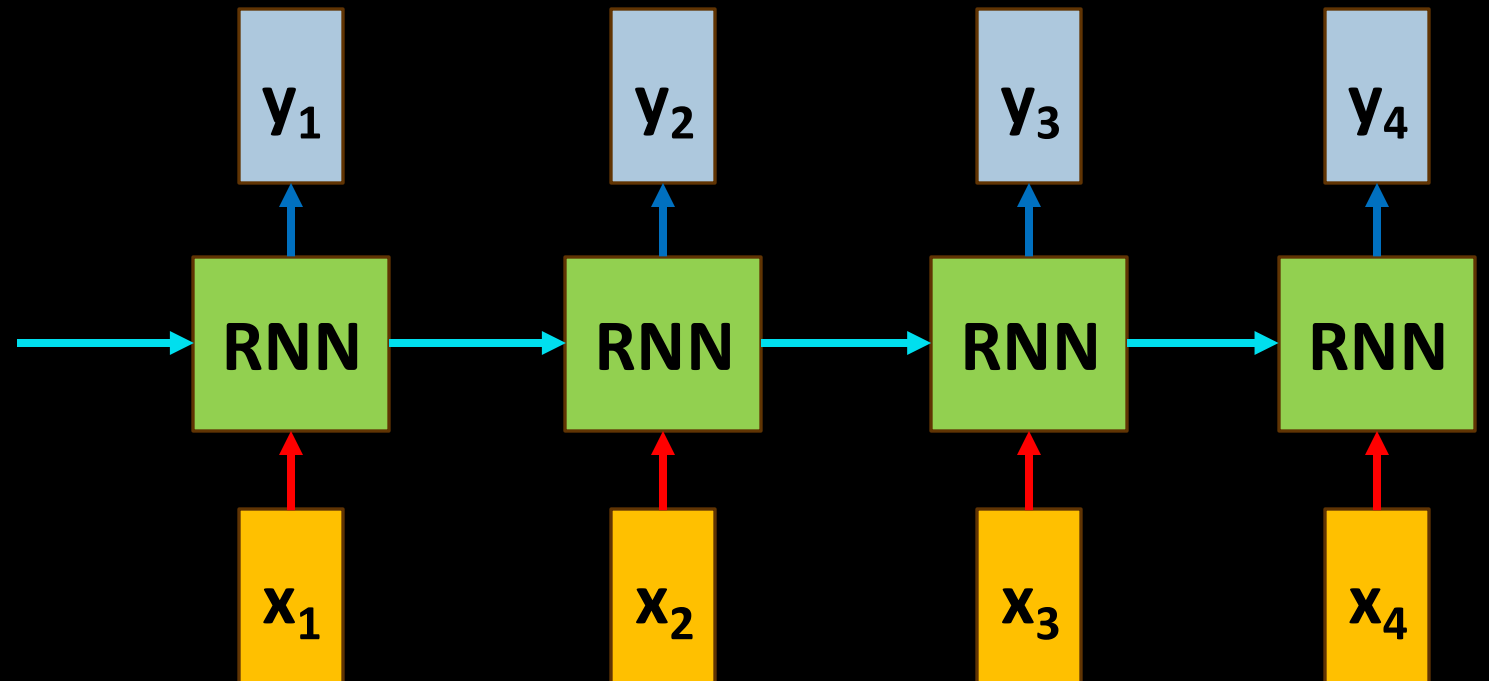
Recurrent Neural Network

One big problem is that the more we unroll a recurrent neural network, the harder it is to train



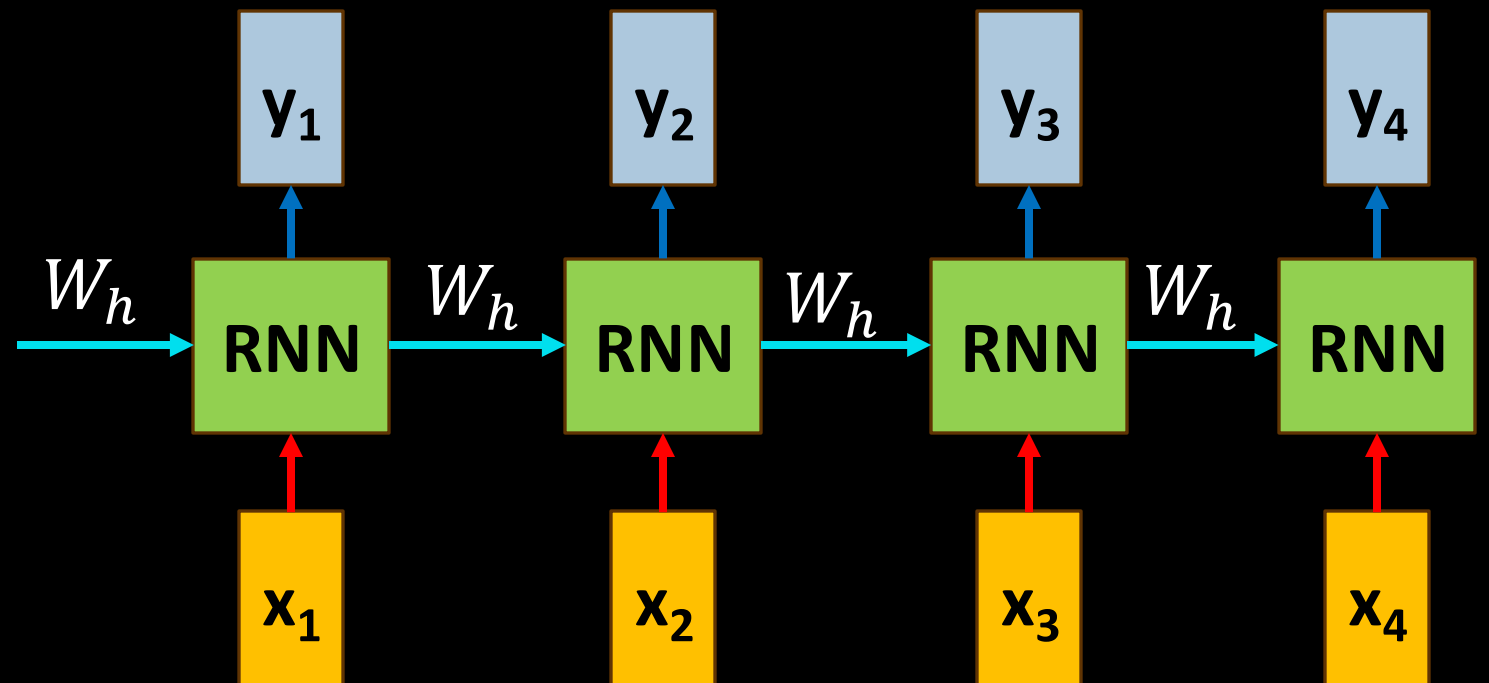
Recurrent Neural Network

This problem is called
**The
Vanishing/Exploding
Gradient Problem.**



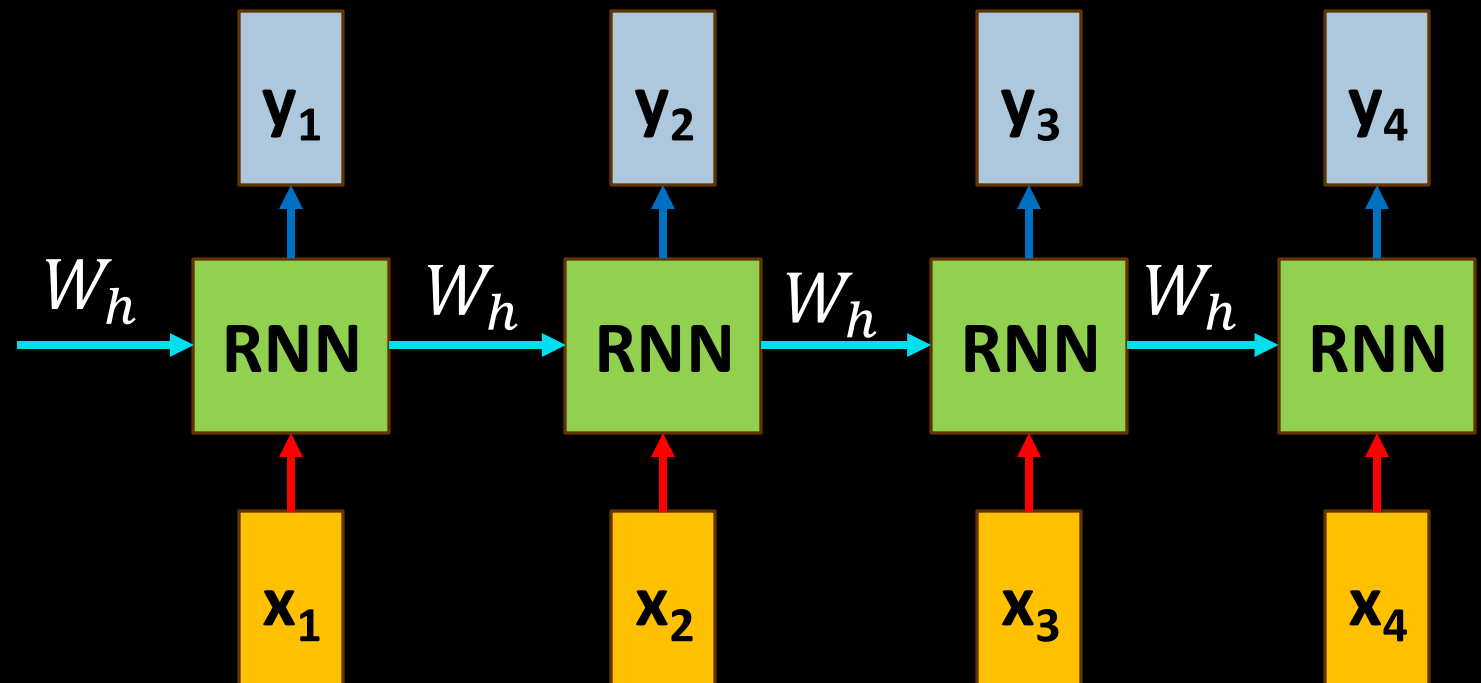
Recurrent Neural Network

In our example the Vanishing/Exploding Gradient problem has to do with the squiggle that we copy each time we unroll the network.



Recurrent Neural Network

Note: To make it easier to understand the Vanishing/Exploding Gradient problem, we're going to ignore the other weights and biases in this network and just focus on W_h



Recurrent Neural Network

Also, just to remind you,
when we optimize
neural networks with
Backpropagation, we
first find the
derivatives, or
Gradients, for each
parameter

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \left(\prod_{t=2}^T \tanh'(W_{hh} h_{t-1} + W_{xh} x_t) \right) W_{hh}^{T-1} \frac{\partial h_1}{\partial W}$$

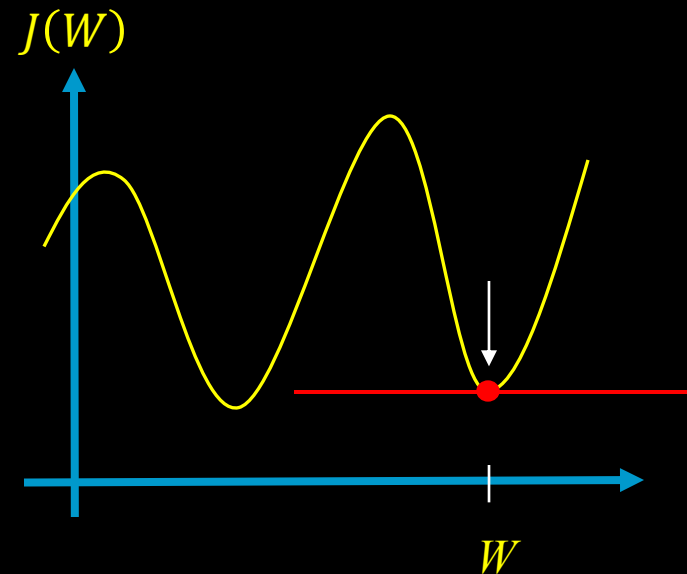
$$W := W - \alpha \frac{\partial J(W)}{\partial W}$$

Recurrent Neural Network

Also, just to remind you, when we optimize neural networks with **Backpropagation**, we first find the derivatives, or Gradients, for each parameter

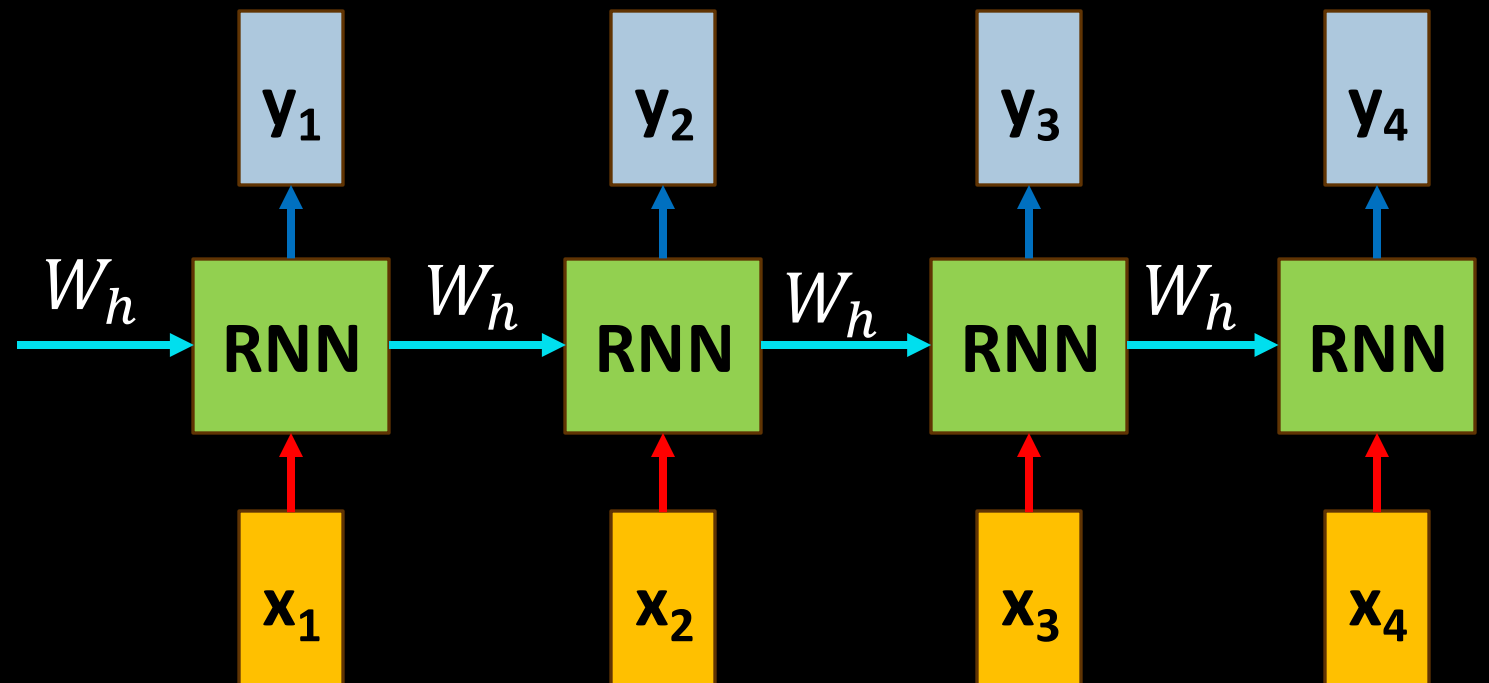
$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \left(\prod_{t=2}^T \boxed{\tanh'(W_{hh} h_{t-1} + W_{xh} x_t)} \right) W_{hh}^{T-1} \frac{\partial h_1}{\partial W}$$

$$W := W - \alpha \frac{\partial J(W)}{\partial W}$$



Recurrent Neural Network

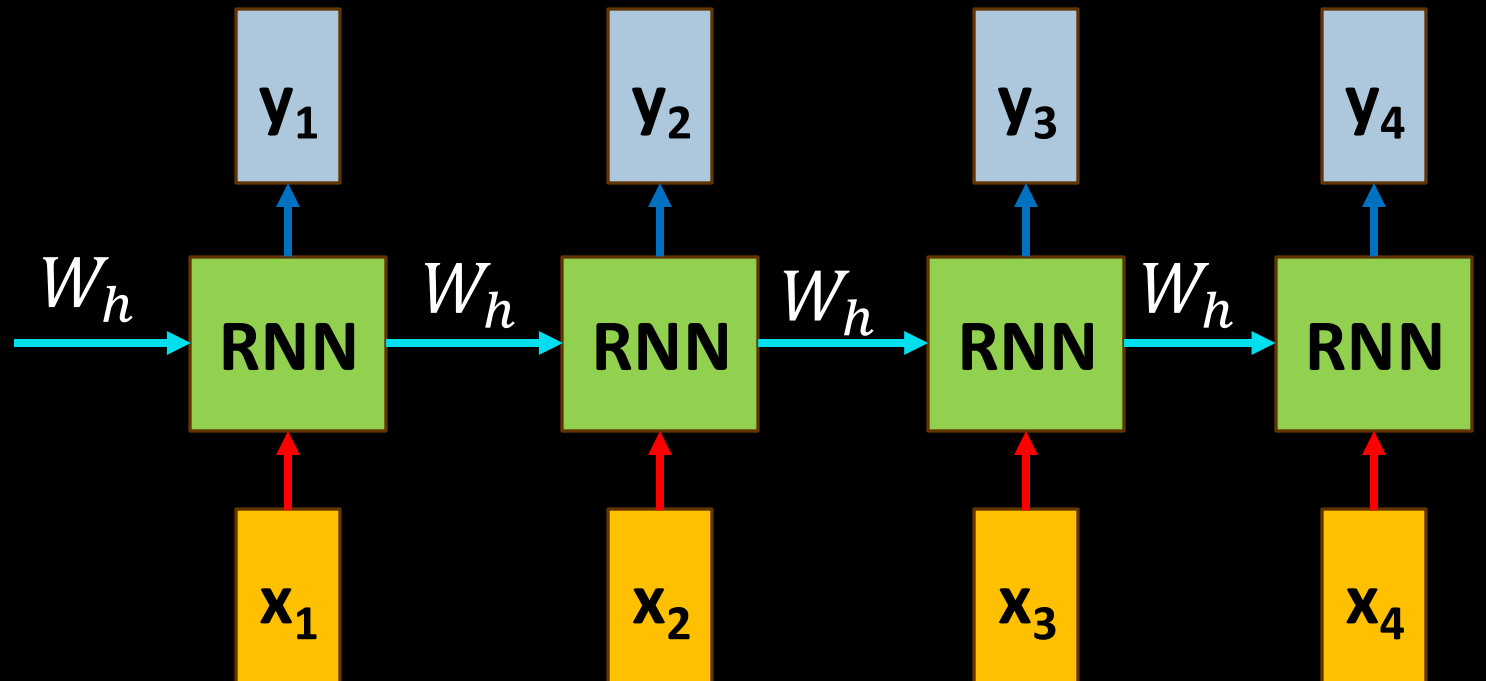
How can a gradient
Explode?



Recurrent Neural Network

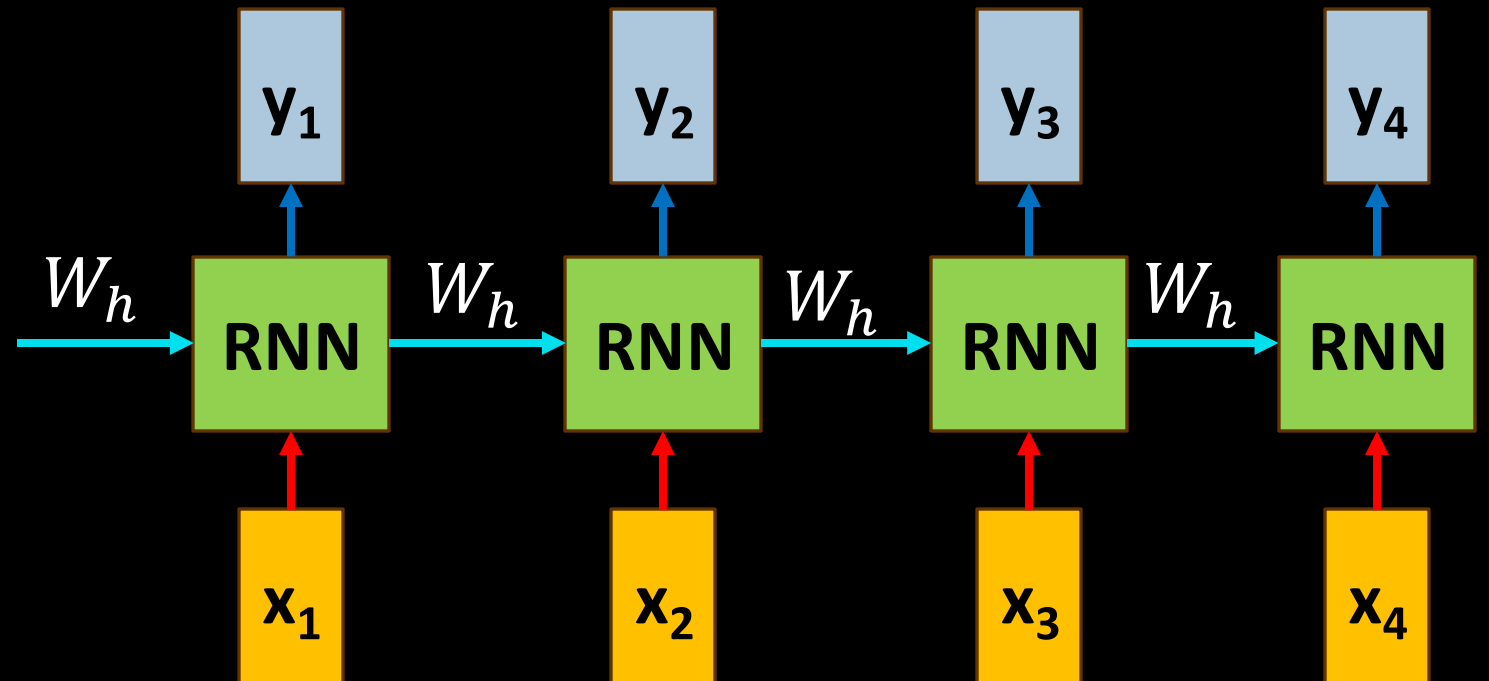
How can a gradient Explode?

In our example, the gradient will explode, when we set W_h to any value larger than 1.



Recurrent Neural Network

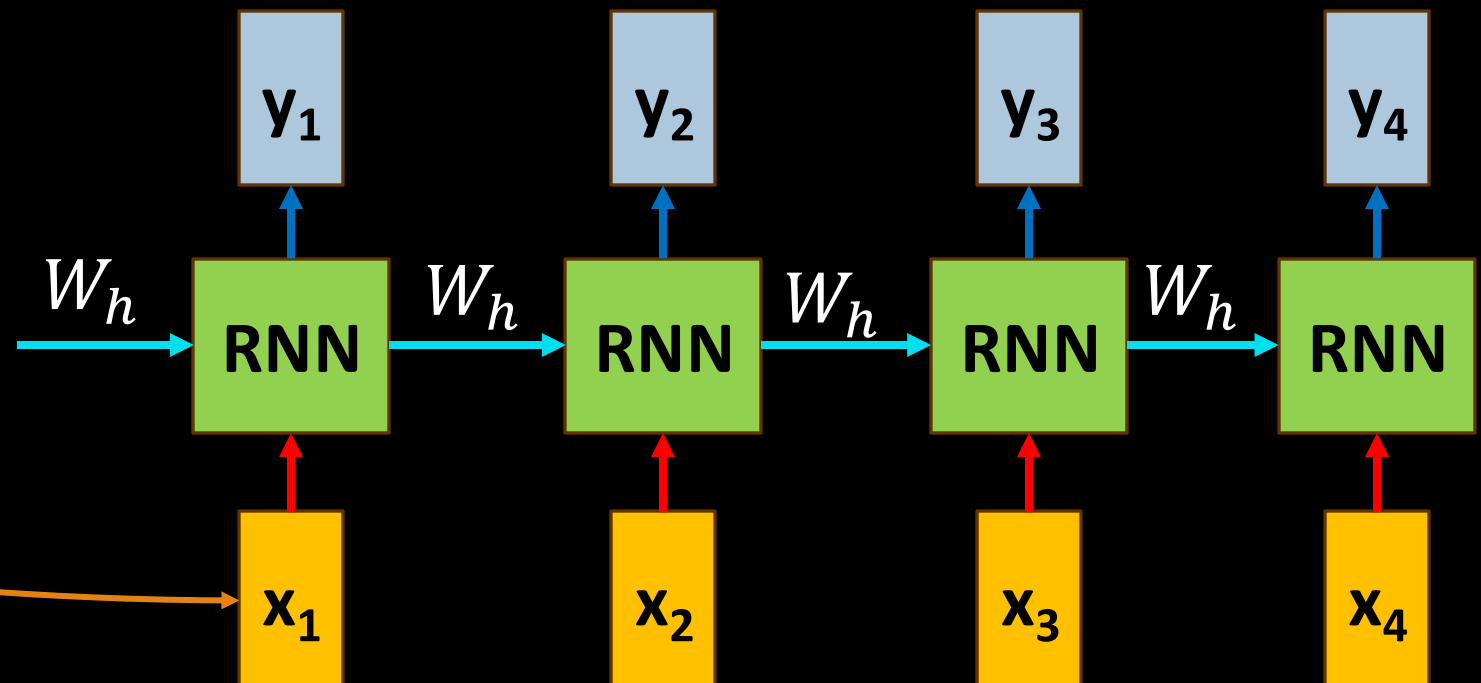
Let's set $W_h = 2$.



Recurrent Neural Network

Let's set $W_h = 2$.

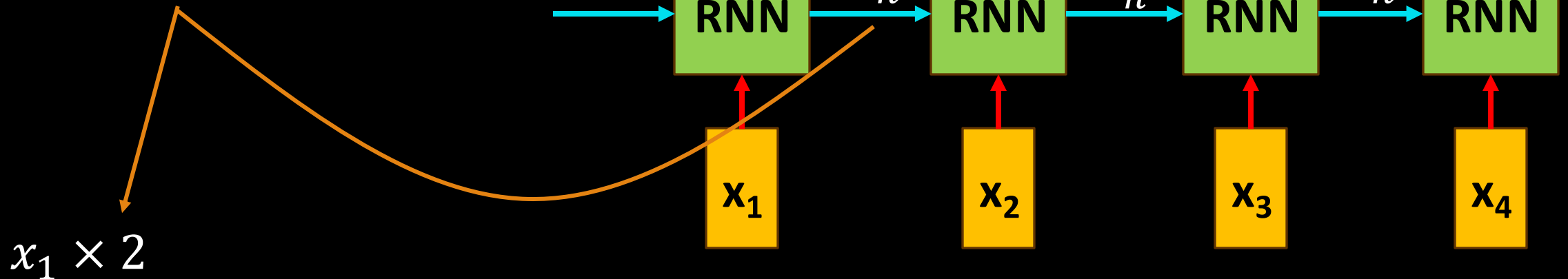
Now, the first input value, $x_1 \dots$



Recurrent Neural Network

Let's set $W_h = 2$.

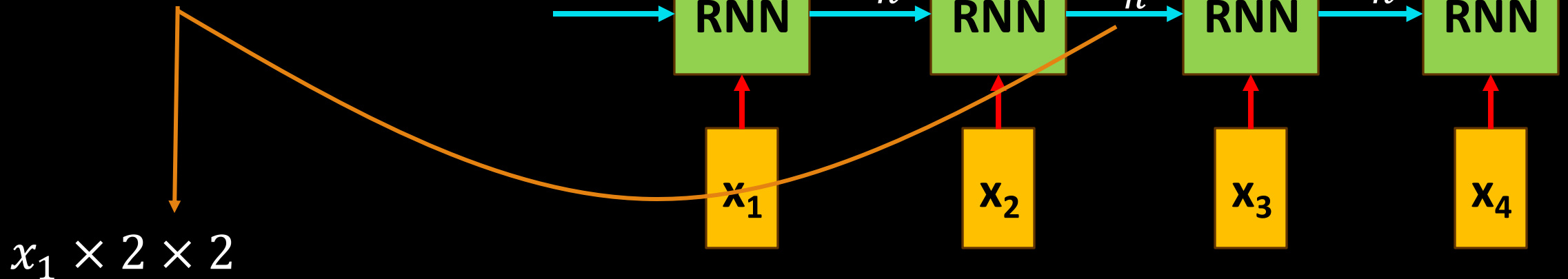
Now, the first input value, $x_1 \dots$



Recurrent Neural Network

Let's set $W_h = 2$.

Now, the first input value, $x_1 \dots$

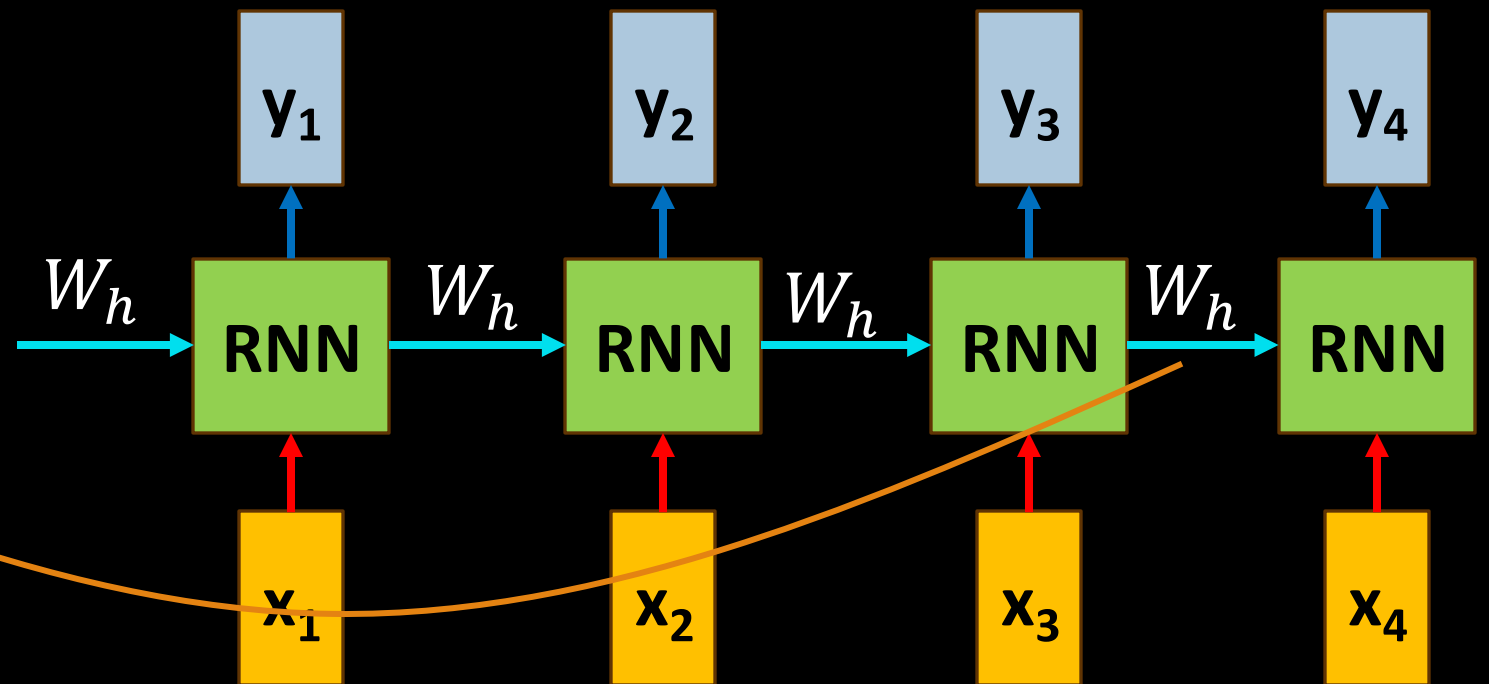


Recurrent Neural Network

Let's set $W_h = 2$.

Now, the first input value, $x_1 \dots$

$x_1 \times 2 \times 2 \times 2$



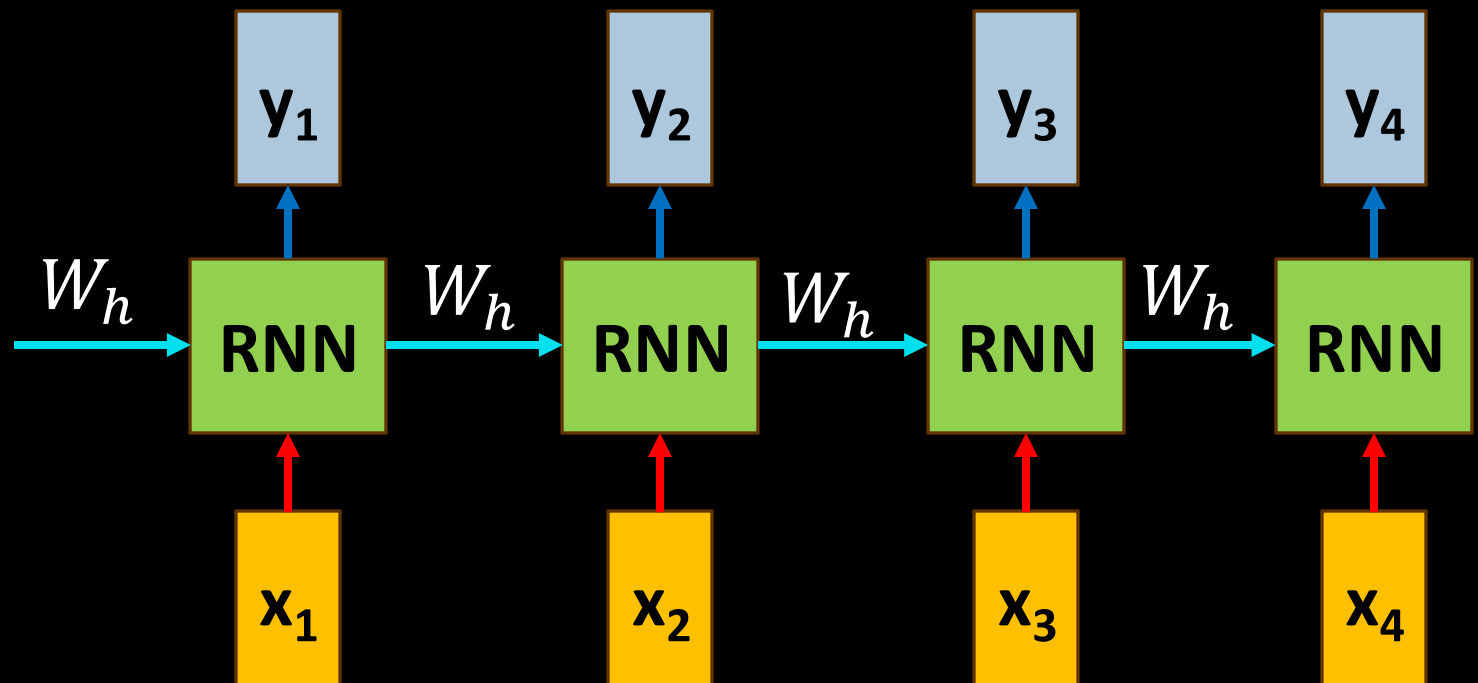
Recurrent Neural Network

We multiply the input value by W_h , which is 2, raised to the number of time we unralled, which is 3

$$x_1 \times 2 \times 2 \times 2$$

$$x_1 \times 2^3$$

$$x_1 \times W_h^{Num.Cell}$$



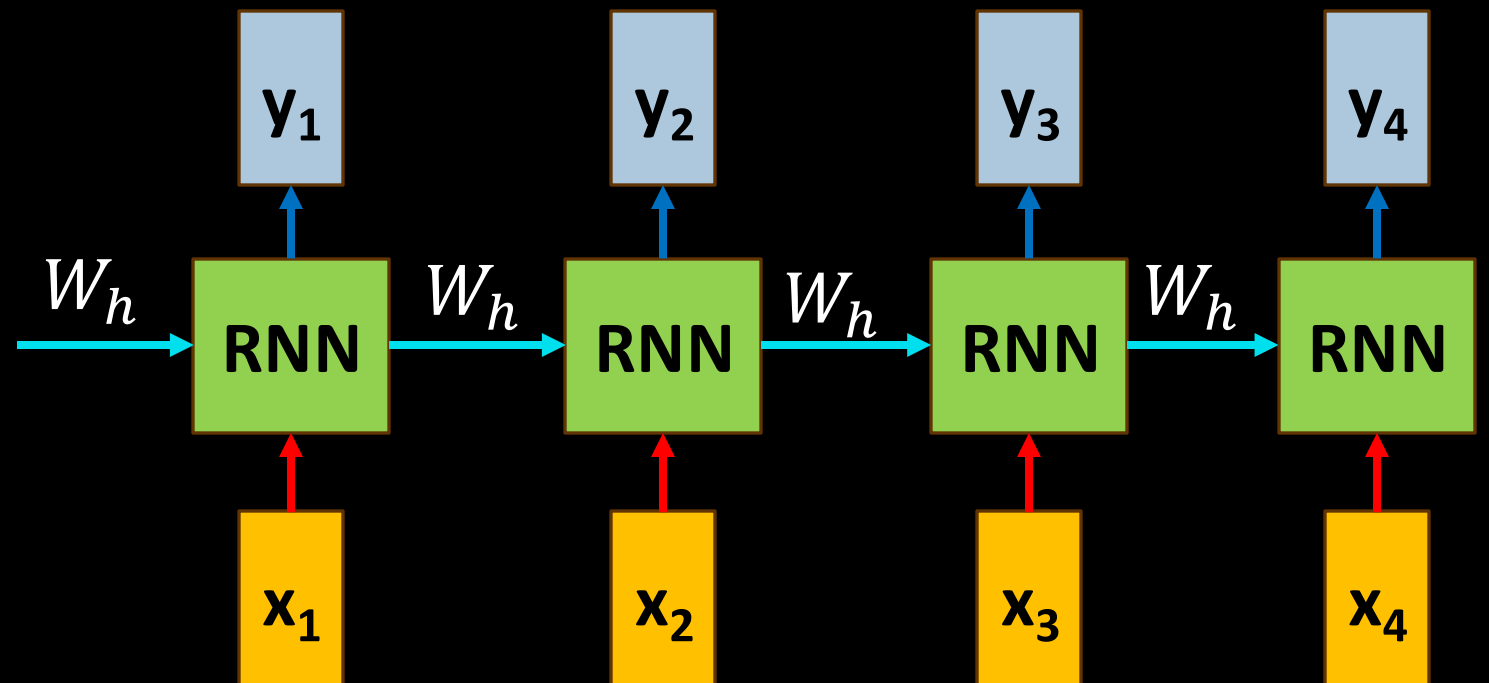
Recurrent Neural Network

And that means the first input value is amplified 8 times before it gets to the final copy of the network

$$x_1 \times 2 \times 2 \times 2$$

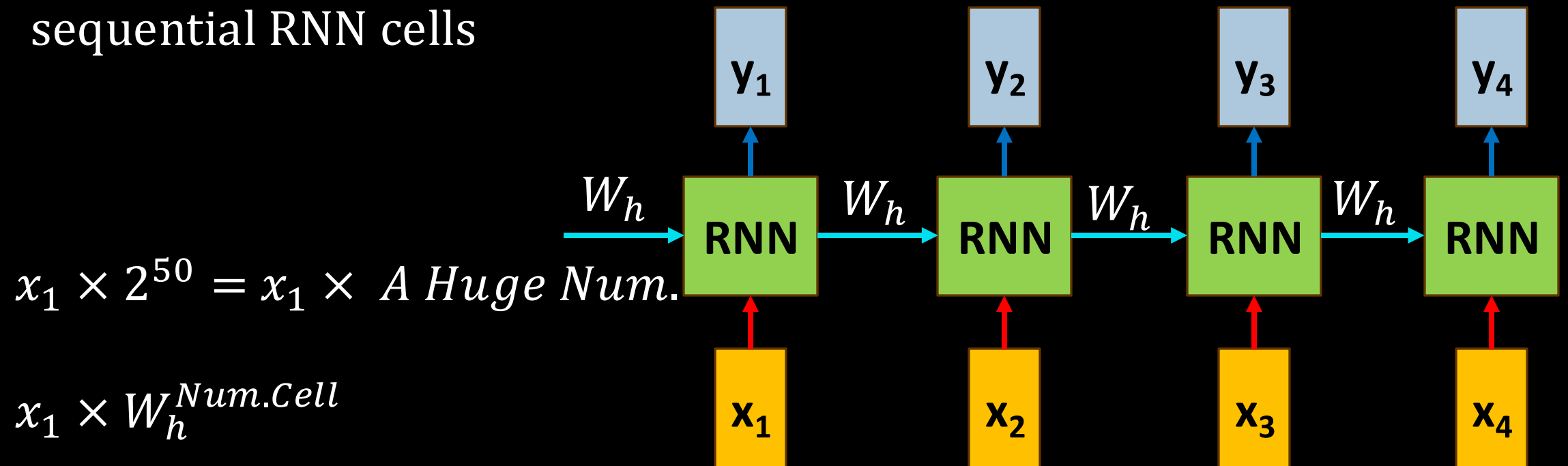
$$x_1 \times 2^3$$

$$x_1 \times W_h^{Num.Cell}$$



Recurrent Neural Network

Now if we had 50
sequential RNN cells

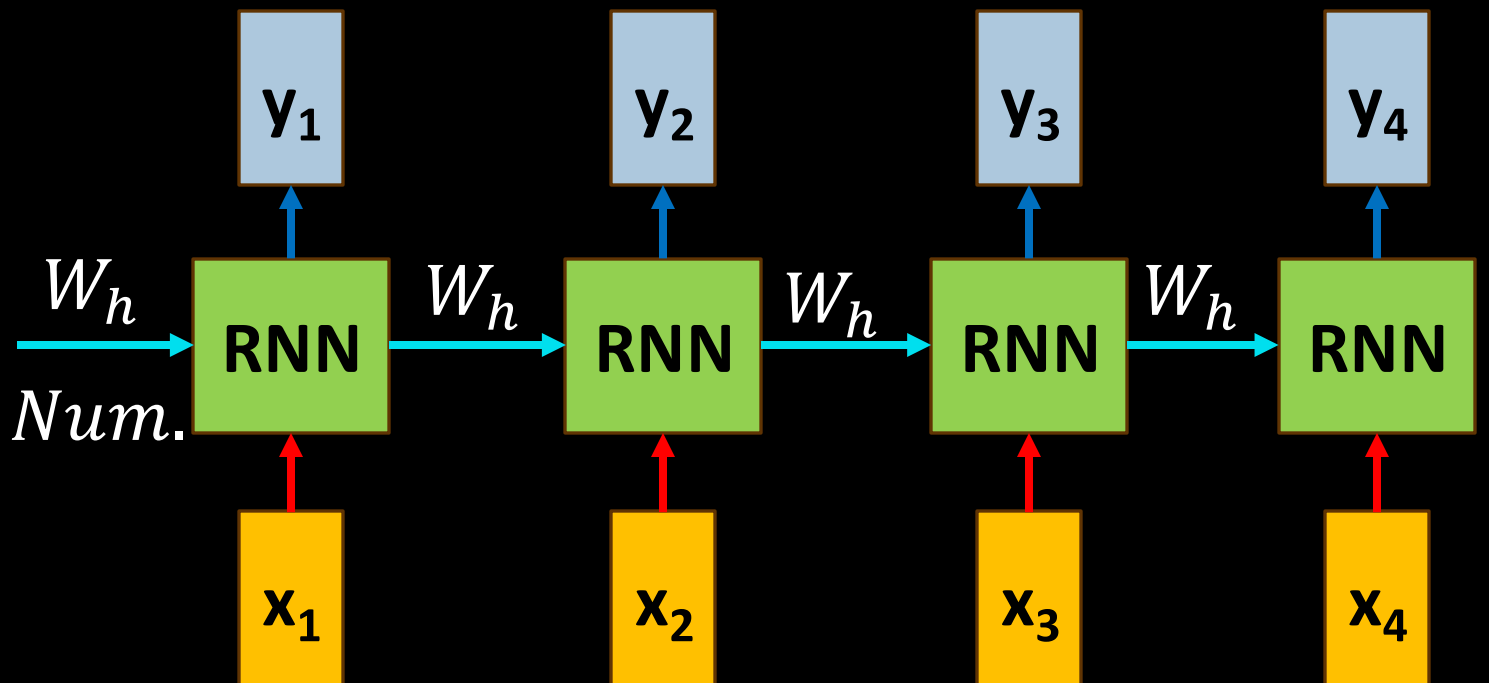


Recurrent Neural Network

This Huge Number is why they call this an **Exploding Gradients Problem**.

$$x_1 \times 2^{50} = x_1 \times \text{A Huge Num.}$$

$$x_1 \times W_h^{Num.Cell}$$



Recurrent Neural Network

If we tried to train this RNN model with backpropagation, this **Huge Number** would find its way into some of the gradients

$$x_1 \times 2^{50} = x_1 \times A \text{ Huge Num.}$$

$$x_1 \times W_h^{Num.Cell}$$

$$\frac{\partial J(W)}{\partial W_x} = \frac{\partial J(W)}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial h_1} \cdot \frac{\partial h_1}{\partial W_x} + \dots$$

$$\frac{\partial h_1}{\partial W_x} = \frac{\partial}{\partial W_x} x_1 \cdot W_x \cdot W_h$$

Recurrent Neural Network

... and that would make it hard to take small steps to find the optimal weights and biases.

$$\frac{\partial J(W)}{\partial W_x} = \frac{\partial J(W)}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial h_1} \cdot \frac{\partial h_1}{\partial W_x} + \dots$$

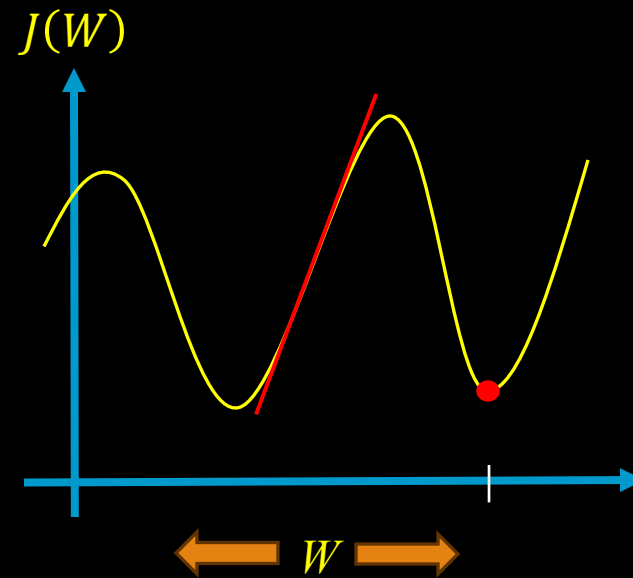
$$\frac{\partial h_1}{\partial W_x} = \frac{\partial}{\partial W_x} x_1 \cdot W_x \cdot W_h$$

Recurrent Neural Network

However, when the gradient contains a huge number, then we'll end up taking relatively large steps.

And instead of finding the optimal parameter, we'll just bounce around a lot

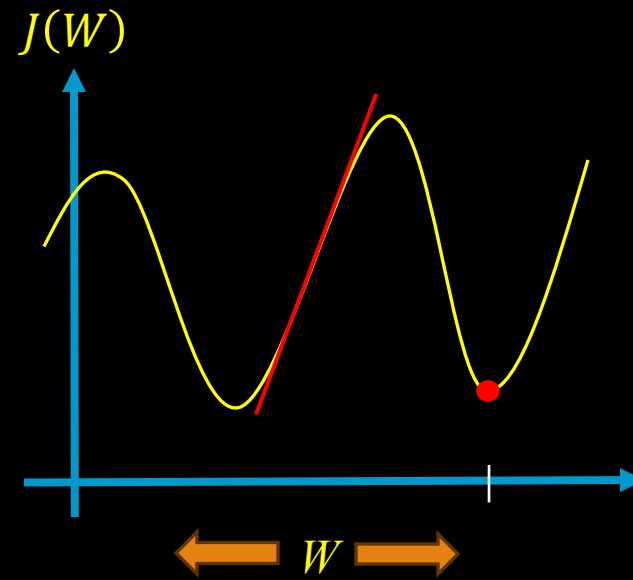
$$W := W - \alpha \frac{\partial J(W)}{\partial W}$$



Recurrent Neural Network

One way to prevent the Exploding Gradient Problem would be to limit W_h to value <1 .

$$W := W - \alpha \frac{\partial J(W)}{\partial W}$$

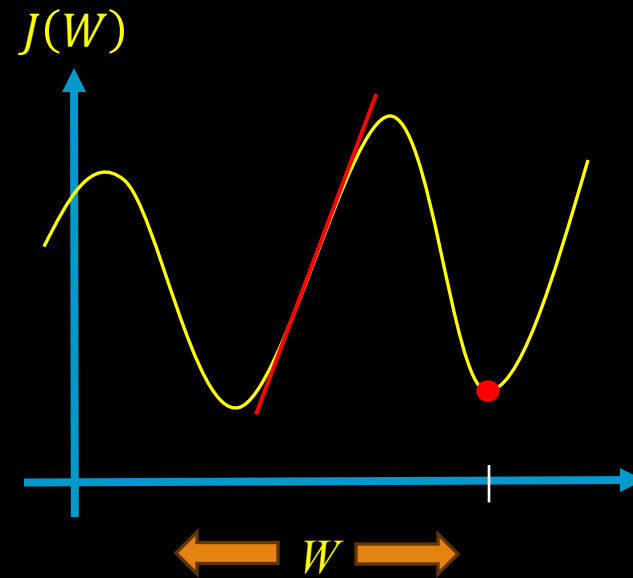


Recurrent Neural Network

One way to prevent the Exploding Gradient Problem would be to limit W_h to value <1 .

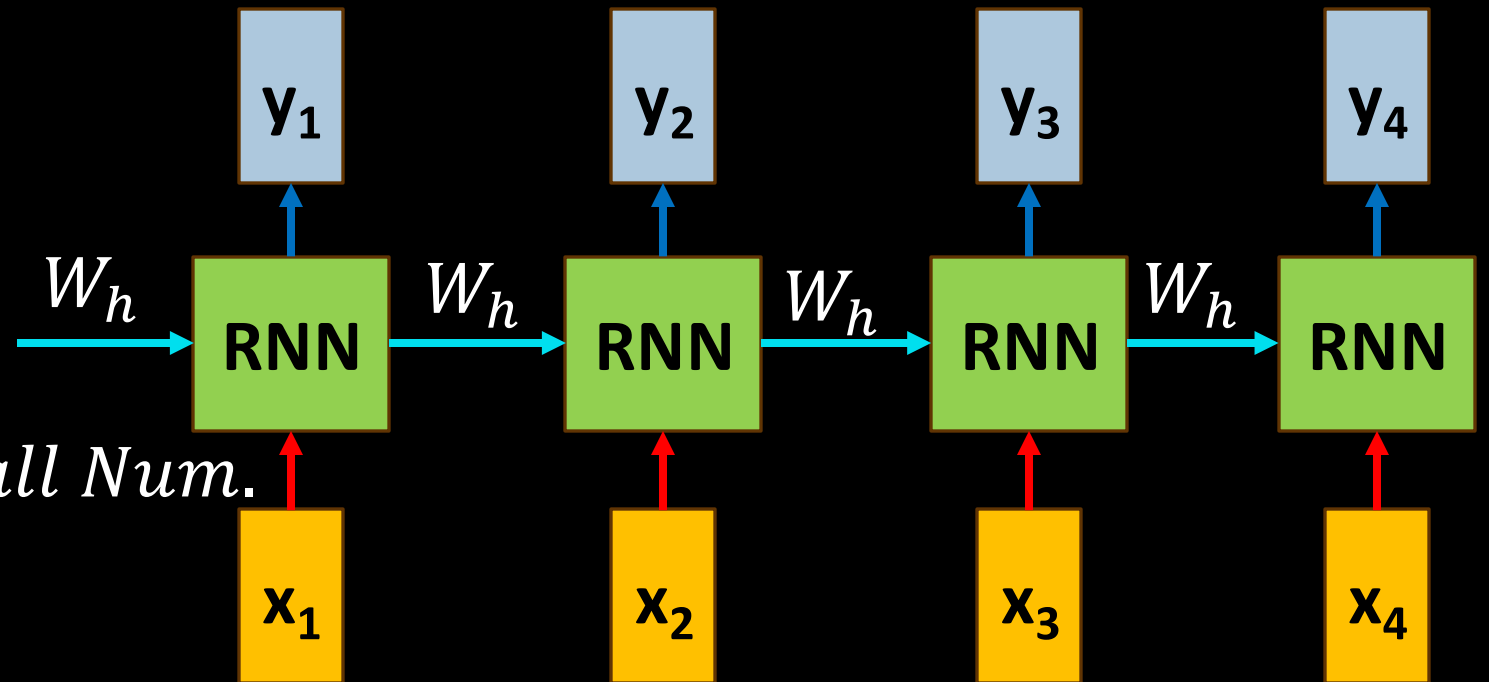
However this results in The **Vanishing Gradient Problem**.

$$W := W - \alpha \frac{\partial J(W)}{\partial W}$$



Recurrent Neural Network

Now if we had 50
sequential RNN cells



$$x_1 \times 0.5^{50} = x_1 \times \text{Very Small Num.}$$

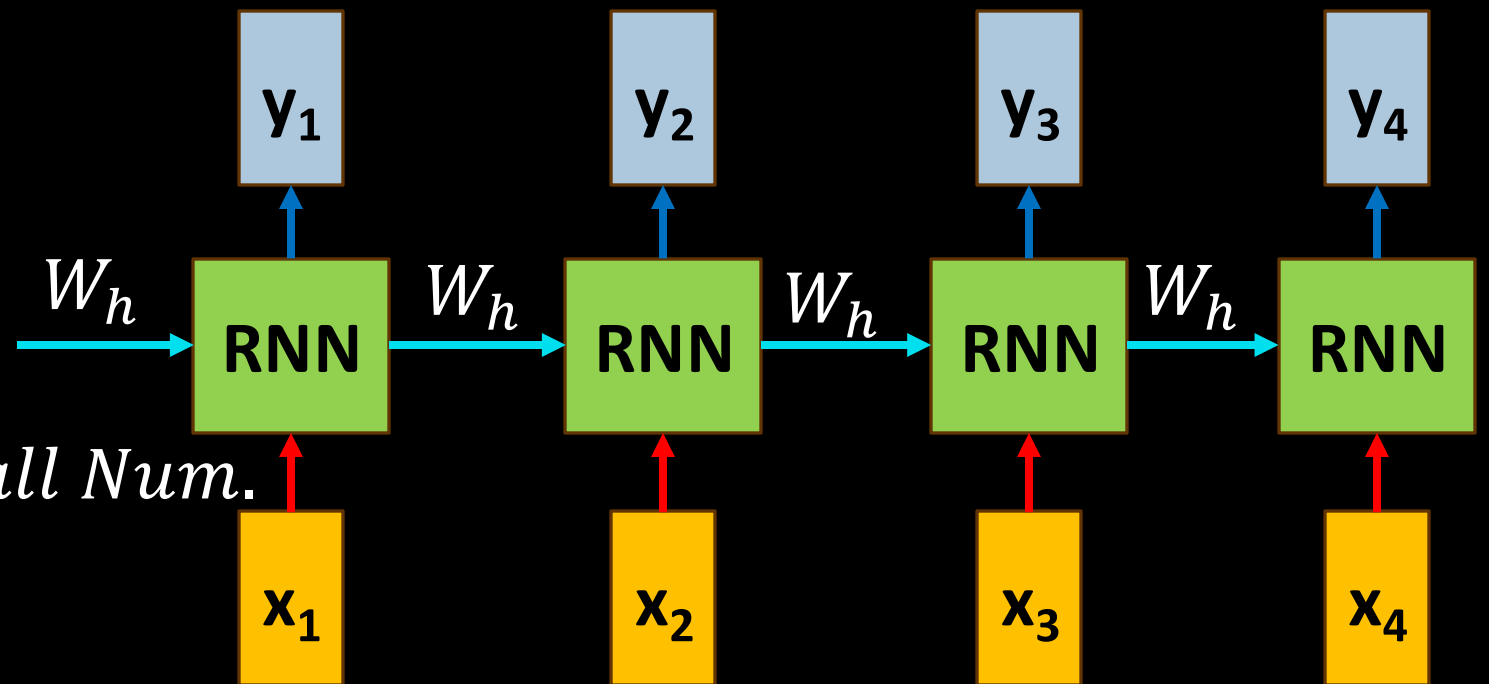
$$x_1 \times W_h^{Num.Cell}$$

Recurrent Neural Network

This number is super close to 0, his is called **Vanishing Gradient Problem**

$$x_1 \times 0.5^{50} = x_1 \times \text{Very Small Num.}$$

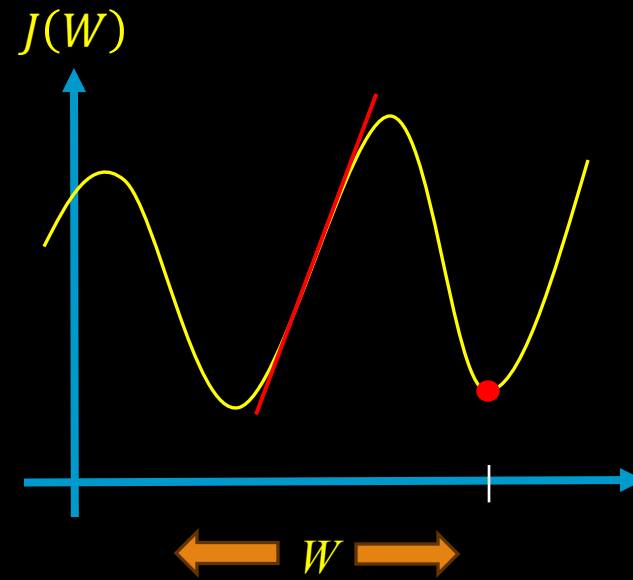
$$x_1 \times W_h^{\text{Num.Cell}}$$



Recurrent Neural Network

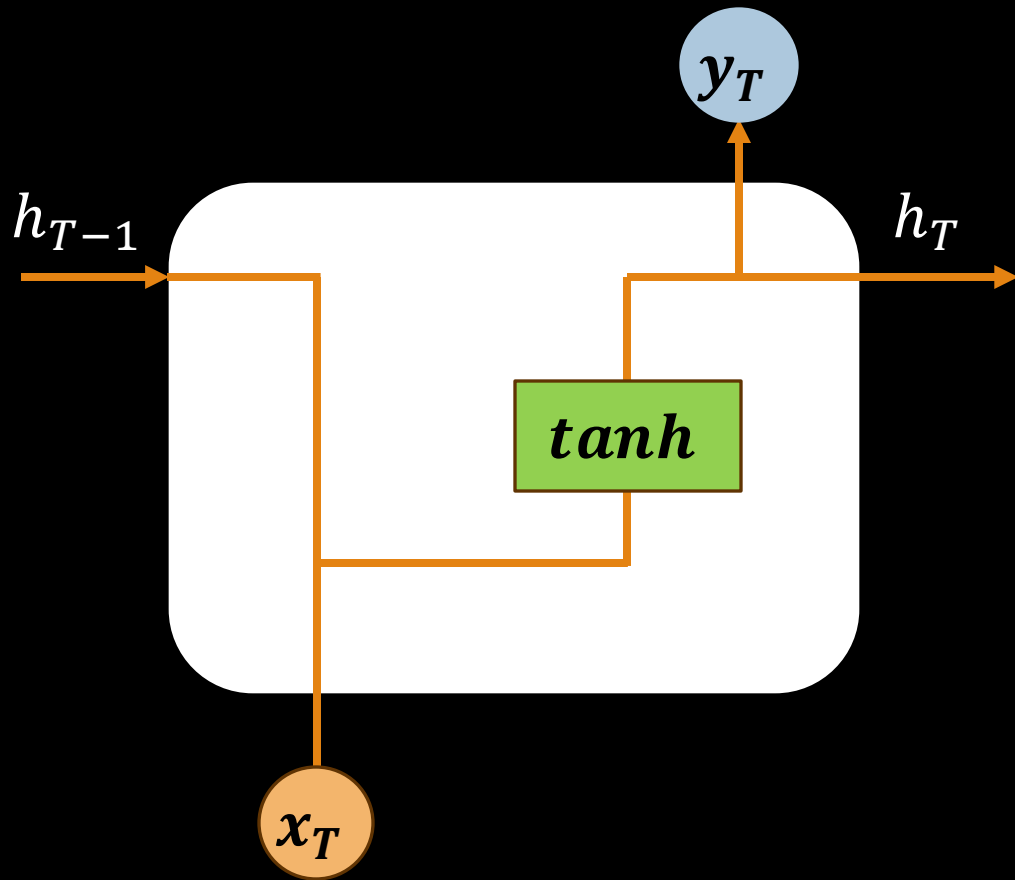
Now, when optimizing a parameter, instead of taking step that are too large, we end up taking steps that are too small.

$$W := W - \alpha \frac{\partial J(W)}{\partial W}$$



Recurrent Neural Network

Summary



$$h_T = f_w(h_{T-1}, x_T)$$

$$h_T = \tanh(W_h h_{T-1}, W_x x_T)$$

$$y_T = f(W_y h_T)$$

Recurrent Neural Network

RNN tradeoffs

RNN Advantages:

- Can process any length of the input
- Computation for step t can (in theory) use information from many steps back
- Model size does not increase for longer input
- The same weights are applied on every timestep, so there is symmetry in how inputs are processed.

RNN Disadvantages:

- Recurrent computation is slow
- In practice, difficult to access information from many steps back

Content

- ~~Recurrent Neural Network (RNN)~~
- Long Short-Term Memory (LSTM)
- Gated Recurrent Unit Network (GRUs)
- Implementation of RNN, GRU and LSTM

Long Short-Term Memory (LSTM)

- **Long short-term memory (LSTM)** is a type of recurrent neural network (RNN) aimed at mitigating the vanishing gradient problem commonly encountered by traditional RNNs.
- Its relative insensitivity to gap length is its advantage over other RNNs, hidden Markov models, and other sequence learning methods.
- It aims to provide a short-term memory for RNN that can last thousands of timesteps (thus "*long* short-term memory").
- The name is made in analogy with long-term memory and short-term memory and their relationship, studied by cognitive psychologists since the early 20th century.

Long Short-Term Memory (LSTM)

Vanilla RNN

$$h_t = \tanh \left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix} \right)$$

LSTM

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

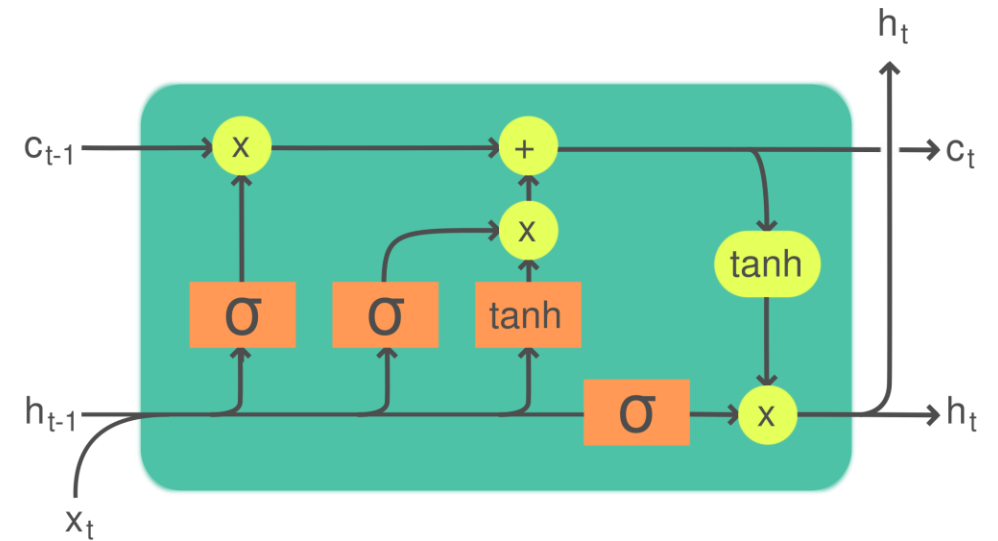
$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

Long Short-Term Memory (LSTM)

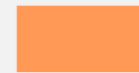
An LSTM unit is typically composed of a cell and **three gates**: an input gate, an output gate, and a forget gate.

The cell remembers values over arbitrary time intervals, and the gates regulate the flow of information into and out of the cell.



Legend:

Layer



Componentwise



Copy



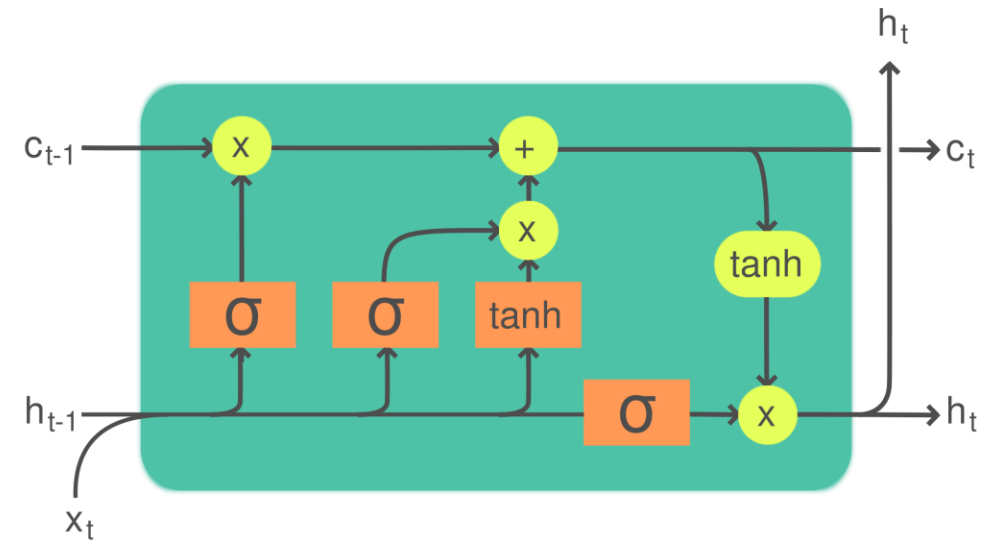
Concatenate



Long Short-Term Memory (LSTM)

Forget gates decide what information to discard from the previous state, by mapping the previous state and the current input to a value between 0 and 1.

A (rounded) value of 1 signifies retention of the information, and a value of 0 represents discarding.

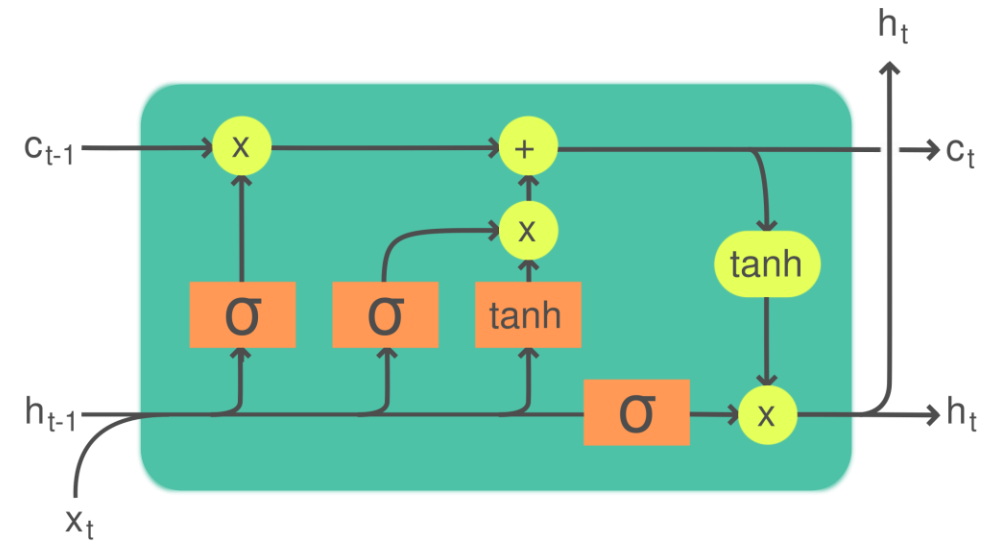


Legend:

Layer	Componentwise	Copy	Concatenate

Long Short-Term Memory (LSTM)

Input gates decide which pieces of new information to store in the current cell state, using the same system as forget gates.



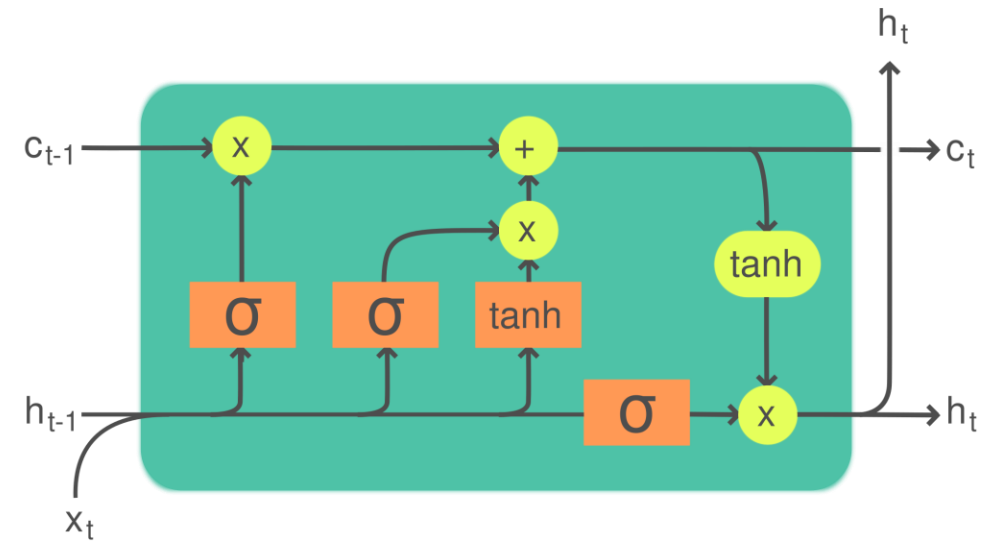
Legend:

Layer	Componentwise	Copy	Concatenate

Long Short-Term Memory (LSTM)

Output gates control which pieces of information in the current cell state to output, by assigning a value from 0 to 1 to the information, considering the previous and current states.

Selectively outputting relevant information from the current state allows the LSTM network to maintain useful, long-term dependencies to make predictions, both in current and future time-steps.

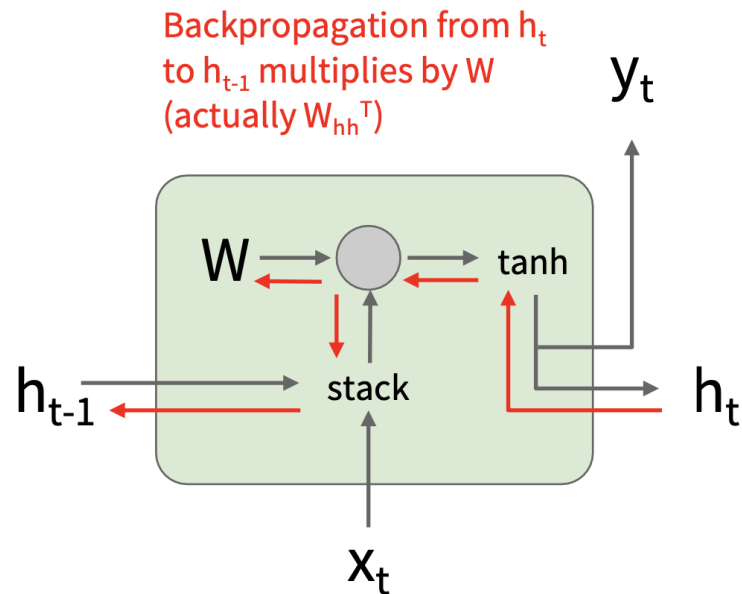


Legend:

Layer	Componentwise	Copy	Concatenate

Long Short-Term Memory (LSTM)

Vanilla RNN Gradient Flow



Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013

$$\begin{aligned}h_t &= \tanh(W_{hh}h_{t-1} + W_{hx}x_t) \\&= \tanh\left((W_{hh} \quad W_{hx}) \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \\&= \tanh\left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right)\end{aligned}$$

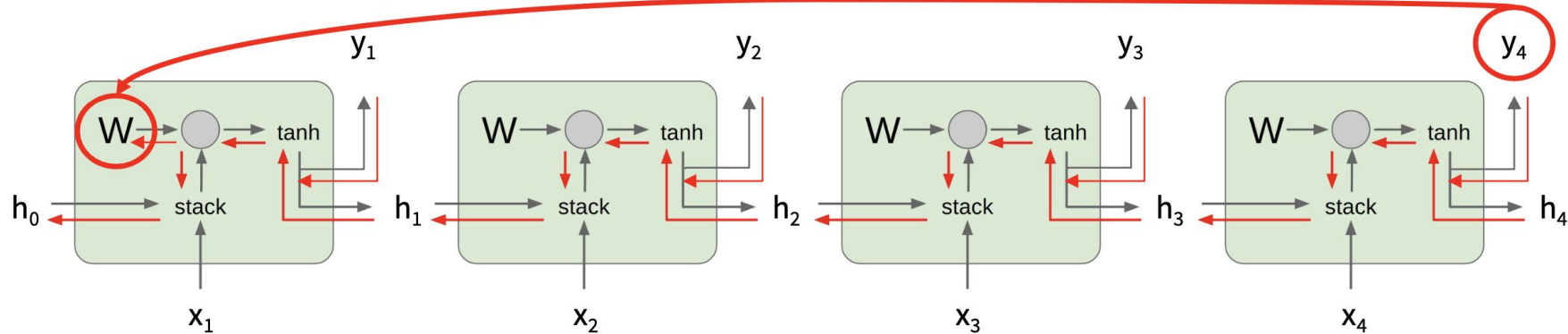
$$\frac{\partial h_t}{\partial h_{t-1}} = \tanh'(W_{hh}h_{t-1} + W_{hx}x_t)W_{hh}$$

Long Short-Term Memory (LSTM)

Vanilla RNN Gradient Flow

Gradients over multiple time steps:

Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013



$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W}$$

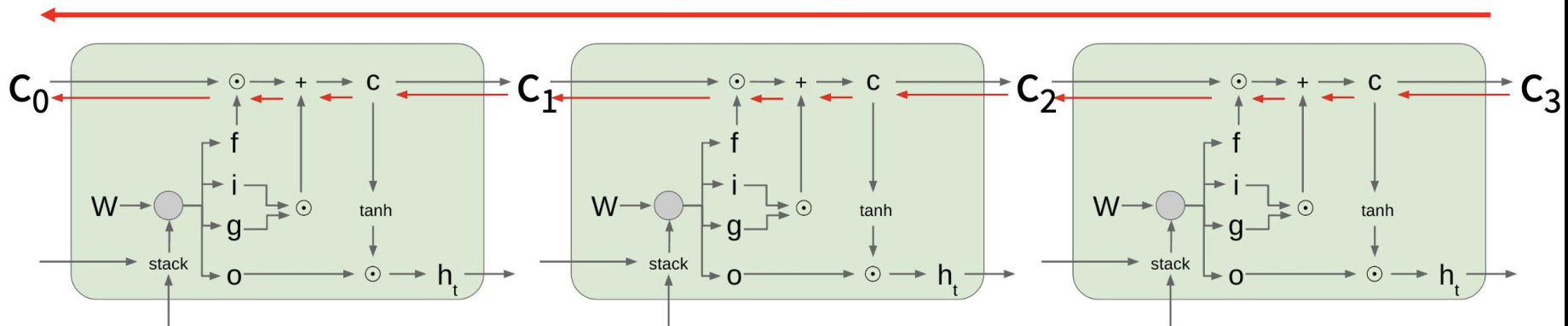
Almost always < 1
Vanishing gradients

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \left(\prod_{t=2}^T \tanh'(W_{hh} h_{t-1} + W_{xh} x_t) \right) W_{hh}^{T-1} \frac{\partial h_1}{\partial W}$$

Long Short Term Memory (LSTM): Gradient Flow

[Hochreiter et al., 1997]

Uninterrupted gradient flow!



Long Short-Term Memory (LSTM)

- The intuition behind the LSTM architecture is to create an additional module in a neural network that learns when to remember and when to forget pertinent information.
- In other words, the network effectively learns which information might be needed later on in a sequence and when that information is no longer needed.
- For instance, in the context of natural language processing, the network can learn grammatical dependencies.

Content

- ~~Recurrent Neural Network (RNN)~~
- ~~Long Short-Term Memory (LSTM)~~
- Gated Recurrent Unit Network (GRUs)
- Implementation of RNN, GRU and LSTM

Gated Recurrent Unit Network (GRUs)

Gated recurrent units (GRUs) are a gating mechanism in recurrent neural networks, introduced in 2014.

The GRU is like a long short-term memory (LSTM) with a **gating mechanism** to input or forget certain features, but lacks a context vector or output gate, resulting in fewer parameters than LSTM.

GRU's performance on certain tasks of polyphonic music modeling, speech signal modeling and natural language processing was found to be similar to that of LSTM.

Gated Recurrent Unit Network (GRUs)

There are several variations on the **full gated unit**, with gating done using the previous hidden state and **light ted recurrent unit**, and a simplified form called **minimal gated unit**.

Gated Recurrent Unit Network (GRUs)

Initially, for $t = 0$, the output vector is $h_0 = 0$.

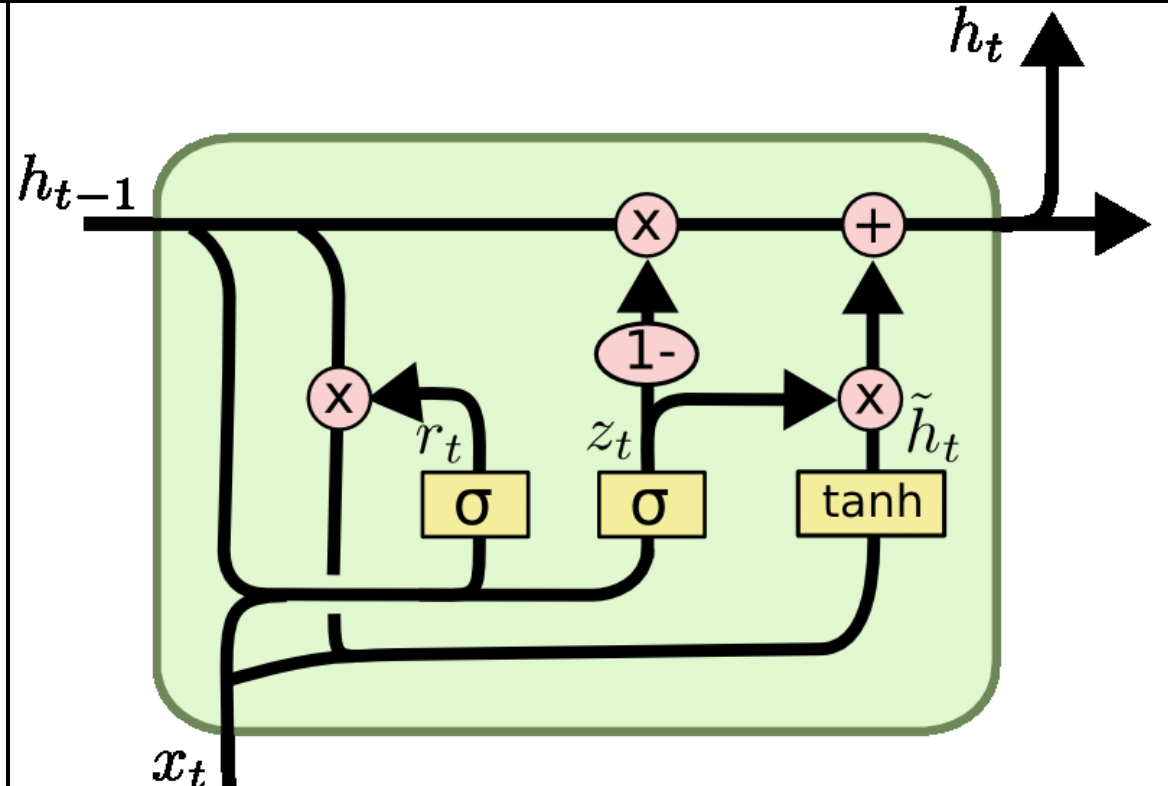
$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z)$$

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r)$$

$$\hat{h}_t = \phi(W_h x_t + U_h (r_t \odot h_{t-1}) + b_h)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \hat{h}_t$$

- $x_t \in \mathbb{R}^d$: input vector
- $h_t \in \mathbb{R}^e$: output vector
- $\hat{h}_t \in \mathbb{R}^e$: candidate activation vector
- $z_t \in (0, 1)^e$: update gate vector
- $r_t \in (0, 1)^e$: reset gate vector
- $W \in \mathbb{R}^{e \times d}$, $U \in \mathbb{R}^{e \times e}$ and $b \in \mathbb{R}^e$: parameter matrices



Content

- ~~Recurrent Neural Network (RNN)~~
- ~~Long Short-Term Memory (LSTM)~~
- ~~Gated Recurrent Unit Network (GRUs)~~
- Implementation of RNN, GRU and LSTM

Implementation of RNN, GRU and LSTM

```
# Function to build models
def build_model(model_type, max_features, max_len):
    model = models.Sequential()
    model.add(layers.Embedding(max_features, 128, input_length=max_len))
    if model_type == 'RNN':
        model.add(layers.SimpleRNN(64, return_sequences=False))
    elif model_type == 'LSTM':
        model.add(layers.LSTM(64, return_sequences=False))
    elif model_type == 'GRU':
        model.add(layers.GRU(64, return_sequences=False))
    model.add(layers.Dense(1, activation='sigmoid'))
    model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
    return model
```


Implementation of RNN, GRU and LSTM

```
# Train, evaluate models, and store results
results = {}
model_types = ['RNN', 'LSTM', 'GRU']
predictions_dict = {}
for model_type in model_types:
    print(f"Training {model_type} model...")
    model = build_model(model_type, max_features, max_len)
    history = model.fit(x_train, y_train, epochs=3, batch_size=64, validation_split=0.2, verbose=2)
    loss, accuracy = model.evaluate(x_test, y_test, verbose=2)
    results[model_type] = accuracy
    predictions = model.predict(x_test)
    predictions_dict[model_type] = predictions
```



